

SIM7600 LTE Backhaul — Operator Manual

What this technology does

The Waveshare SIM7600G-H modem is Gateway-01's commissioned **production-primary Internet path**. The gateway uses the QMI data path with control device `/dev/cdc-wdm0` and network interface `wwan0`.

The design intentionally keeps management Ethernet separate from production traffic so field tests measure the real cellular path.

Current routing contract

- LTE interface: `wwan0`.
- QMI control device: `/dev/cdc-wdm0`.
- LTE is health-gated production primary.
- Wi-Fi is automatic fallback.
- Ethernet is management-only.
- `/usr/sbin/lte-route-health` owns the production default-route policy.
- Normal LTE default route metric: 10.
- Production MQTT traffic must resolve through LTE in normal commissioned state.

Do not manually force a permanent route through management Ethernet to make a cloud test pass.

Step 1 — Confirm the modem exists

Run on Gateway-01:

```
lsusb 2>/dev/null || true
ls -l /dev/cdc-wdm* /dev/ttyUSB* 2>/dev/null || true
ip link show wwan0 2>/dev/null || true
dmesg | grep -Ei 'simcom|sim7600|qmi|cdc-wdm|wwan|usb' | tail -n 80
```

PASS means: the SIMCom/Waveshare USB device is present, `/dev/cdc-wdm0` exists, and `wwan0` exists.

If the modem repeatedly disconnects/re-enumerates: check power and USB stability before changing APN or routes.

Step 2 — Check the logical LTE interface

```
ubus call network.interface.lte status
```

PASS means: the interface reports up with a current address/gateway appropriate to the carrier session.

Do not reuse an old observed carrier IP as a static configuration value. Cellular PDP addresses can change after reconnect/reboot.

Step 3 — Check the route controller

```
ubus call service list '{"name":"lte-route-health"}'  
logread | grep 'lte-route-health' | tail -n 40
```

PASS means: the controller is running and not stuck in a rapid restart/reset loop.

The commissioned controller has a boot grace and staged recovery. Immediately after boot, allow it to converge rather than injecting manual routes.

Step 4 — Verify route selection

```
ip route show default  
ip route get 129.212.208.168
```

PASS means: during normal commissioned LTE operation the production destination resolves over `wwan0`, and the preferred production default is the health-managed LTE route (normally metric 10).

If public endpoint addressing changes intentionally, verify the currently configured production MQTT hostname/IP from the gateway bridge configuration instead of assuming the historical address above.

Step 5 — Verify name resolution and UTC

```
nslookup smartagri-mqtt.duckdns.org  
date -u
```

A bad clock can make valid TLS certificates look expired/not-yet-valid. Fix the network/time path before replacing certificates.

Step 6 — Verify production TCP/TLS reachability

Do not print the private key. Inspect the bridge metadata first:

```
grep -E '^(connection |address |bridge_cafile|bridge_certfile|bridge_keyfile|bridge_insecure|topic )'  
/etc/mosquitto/conf.d/bridge.conf
```

Then verify the actual bridge is established using service logs/socket state rather than creating a second competing MQTT session:

```
logread | grep -Ei 'mosquitto|bridge|connected|connack|tls|certificate' | tail -n 80  
ss -tnp 2>/dev/null | grep ':8883' || netstat -tnp 2>/dev/null | grep ':8883' || true
```

PASS means: the gateway's configured cloud bridge has an established TLS/MQTT path and the route to that destination is via the intended production interface.

Step 7 — Prove real traffic, not only connectivity

Generate one approved sensor uplink and verify in order:

1. local gateway MQTT event appears;

2. cloud broker sees the gateway event;
3. ChirpStack accepts the uplink;
4. application telemetry appears.

If local event passes but cloud delivery fails, the fault is in LTE/routing/DNS/TLS/MQTT—not RF.

Step 8 — What to do during intermittent mobile service

Normal behavior is:

```
RF continues
-> local MQTT accepts/buffers QoS-1 uplinks
-> LTE controller keeps/re-establishes a usable data path
-> cloud bridge reconnects
-> queued uplinks drain
```

Do not purge `mosquitto.db`, reset the gateway lineage, or route production traffic over management Ethernet simply because the carrier is temporarily weak.

For research resilience tests, use the dedicated test procedure. Do not improvise an outage by repeatedly unplugging the modem during ordinary troubleshooting.

Change procedure for APN/modem/network settings

1. keep management Ethernet connected;
2. back up Gateway OS configuration;
3. record current `ubus call network.interface.lte status`, `ip route`, and controller log state;
4. change only the required carrier/APN/network item;
5. verify modem registration and QMI addressing;
6. allow `lte-route-health` to converge;
7. verify DNS and UTC;
8. verify route to production MQTT;
9. verify TLS/MQTT bridge;
10. verify one real uplink end to end.

Do not send USB-mode-changing AT commands to the commissioned SIM7600 unless a separately reviewed hardware replacement/recovery procedure requires it.

Troubleshooting order

Symptom	Inspect first
No USB device	power, cable/port, modem hardware
USB present, no <code>/dev/cdc-wdm0/wwan0</code>	Gateway OS modem driver/image
<code>wwan0</code> exists, no data session	SIM, carrier registration, APN/PDP
Data session works, route wrong	<code>lte-route-health</code> , route metrics/policy
Route works, DNS/time broken	resolver/NTP/general Internet

Symptom	Inspect first
TCP works, TLS fails	UTC, CA/SAN/client cert, server identity
TLS/MQTT works, app missing	ChirpStack/application path

Recovery

If the dataplane remains unavailable, use the commissioned staged recovery path rather than manually flapping everything at once. The controller can perform a bounded logical-interface restart and, only after cooldown when required, a SIM7600 functional reset. If the modem is physically unstable, solve power/USB first.

Rebuilding the Gateway OS is a last resort for a driver/image regression and must preserve the known-good rollback image/config/identity boundary.

Completion checklist

- SIM7600 detected.
- `/dev/cdc-wdm0` and `wwan0` present.
- LTE logical interface has a live session.
- `lte-route-health` is active.
- production route uses LTE during normal operation.
- DNS and UTC are healthy.
- MQTT bridge is connected over TLS.
- a real uplink reaches ChirpStack/application through the cellular path.

Detailed runbooks

- [../server/cloud-production/11-raspberry-pi-4g-backhaul.md](#)
- [../gateway/setup/02a-build-sim7600-capable-gateway-os.md](#)
- [../gateway/setup/06-verify-gateway-os.md](#)
- [../gateway/operations/03-availability-tests.md](#)

Supporting runbook: 11-raspberry-pi-4g-backhaul.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

11. Gateway OS Delivery Buffer and Integrity Journal over a USB 4G/LTE Dongle

Current status: BUILD/LTE/CLOUD/RF NORMAL-PATH PASS; LTE IS THE COMMISSIONED FIELD UPLINK. Gateway-01 uses the SIM7600 QMI wwan0 path for normal production Internet traffic, while RJ45/br-1an at 192.168.20.11 is management-only. The logical LTE interface deliberately keeps defaultroute=0; /usr/sbin/lte-route-health owns the metric-10 production default. On 2026-09-15 a live preflight exposed a stale-clock condition caused by an older route-manager policy that could remove the LTE default after a failed MQTT/TLS probe even though the DITO PDP session itself was healthy. The commissioned controller was corrected so a valid QMI address+gateway installs the LTE default **before** external probing and keeps that route available for DNS, NTP, MQTT, and autonomous recovery even if the broker probe is temporarily unavailable. A controlled reboot then proved unattended recovery: DITO re-registered automatically, assigned 100.82.220.182/30 with gateway 100.82.220.181, the daemon restored default via 100.82.220.181 dev wwan0 ... metric 10, and the production MQTT mTLS probe passed without a manual route command. NTP recovered over LTE, and the post-reboot research-recorder preflight exited 0 with CLOCK_GATE=PASS, gateway/cloud skew -0.153889 s, cloud spread 0.004709 s, all three ULC nodes reachable, evidence verification 70 with zero pending/gaps/integrity failures, and EMU/SEC fixtures present. Earlier lte_4 DHCP-child incidents in this document are historical troubleshooting evidence from the pre-direct-QMI commissioning path, not the current runtime design. Provider Reserved-IP failover authority and external Fabric execution remain separate acceptance boundaries. **Do not perform LTE-outage, broker-loss, or queue-drain failure experiments here; the dedicated resilience phase owns those tests.**

11.0 Fast reuse gate - inspect before changing the gateway

Do not reflash Gateway OS or rewrite a working radio configuration merely because Phase 11 is now active. First capture the current gateway state in one read-only pass. Reuse any component that already satisfies the commissioned requirements and continue only from the first missing item.

The reuse gate must identify: Gateway OS release, UTC time, free persistent storage, active RAK5146/SX1303 Concentrator configuration, authoritative Gateway EUI, plain as923 topic prefix, UDP Forwarder disabled state, MQTT Forwarder endpoint/backend, any existing loopback Mosquitto listener/configuration, and USB/LTE interfaces. A previous working result is useful context but does not replace this current inspection.

Current LTE hardware baseline: commissioned Gateway-01 uses a **Waveshare SIM7600G-H 4G DONGLE** in a verified QMI composition with control device /dev/cdc-wdm0 and production data interface wwan0. Preserve that working composition and do not send USB-mode-changing AT commands. The broader lsusb/kernel//dev/ttyUSB*//dev/cdc-wdm* discovery procedure below remains useful only when commissioning a

replacement gateway/dongle or when the live composition genuinely differs; it is not permission to reinterpret the known-working Gateway-01 runtime away from QML.

Why: this shortens the phase while reducing risk. Reflashing or rewriting a functioning gateway would create unnecessary RF identity, networking, and credential changes. Read-only discovery lets the operator skip already-complete setup and mutate only the missing layer.

11.0A Fast read-only gateway inventory

Run this directly on the Gateway OS shell. It intentionally avoids full wireless/network secret dumps and never reads a private-key body:

```
echo '=== PHASE 11.0A GATEWAY REUSE INVENTORY ==='

echo '--- OS / time / storage ---'
cat /etc/os-release
uname -a
date -u
df -h /

echo '--- core service state ---'
monit status 2>/dev/null || true
ps w | grep -E '[c]hirpstack-concentrator|[c]hirpstack-mqtt-forwarder|[m]osquitto'

echo '--- Concentrator effective config ---'
uci show chirpstack-concentrator 2>/dev/null || true

echo '--- recent Concentrator identity/startup ---'
logread -e chirpstack-concentrator 2>/dev/null | tail -n 120

echo '--- MQTT Forwarder effective config ---'
uci show chirpstack-mqtt-forwarder 2>/dev/null || true

echo '--- UDP Forwarder effective config ---'
uci show chirpstack-udp-forwarder 2>/dev/null || true

echo '--- local Mosquitto process/listener ---'
ps w | grep '[m]osquitto' || true
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp 2>/dev/null | grep ':1883' || true

echo '--- Mosquitto persistence/listener policy ---'
grep -E
'^ (persistence|persistence_location|persistence_file|autosave_interval|max_queued_messages|max_queued_bytes|l
istener|protocol|allow_anonymous|include_dir)' /etc/mosquitto/mosquitto.conf 2>/dev/null || true

echo '--- Mosquitto cloud bridge metadata only ---'
grep -E '^ (connection |address
|remote_clientid|cleansession|bridge_cafile|bridge_certfile|bridge_keyfile|bridge_insecure|topic )'
/etc/mosquitto/conf.d/bridge.conf 2>/dev/null || true

echo '--- network / USB / LTE discovery ---'
ip link
ip -4 addr
ip route
lsusb 2>/dev/null || true
dmesg | tail -n 120

echo '=== PHASE 11.0A COMPLETE ==='
```

Record the active 16-hexadecimal Gateway EUI from the successful SX1302/SX1303 Concentrator startup, not an inactive SX1301 placeholder. The current POC region topic prefix must remain exactly as 923. A correct already-buffered delivery path has MQTT Forwarder targeting tcp://127.0.0.1:1883 with QoS 1, UDP Forwarder disabled, local Mosquitto bound only to loopback 1883, persistence enabled with finite queue limits, and cloud bridge metadata referencing the intended MQTT endpoint. If any of those are already correct, do not recreate them.

The inventory may show an LTE device/interface but does not prove carrier registration or cloud reachability; those are verified later after the modem mode is identified. If Mosquitto or the bridge files are absent, record ABSENT and continue with the local-buffer section rather than installing packages blindly.

Phase 11.0A/0B live inventory result - 2026-08-26

PASS for gateway reuse / PARTIAL for LTE driver binding. The gateway is running ChirpStack Gateway OS 4.12.0 (r29197-ab4c7d6af7) on Raspberry Pi 4B with about 3.8 GiB free on the persistent overlay. The active Concentrator is chirpstack-concentrator-sx1302 4.7.1 using model='rak_5146', region AS923, channel plan as923, with active Gateway EUI 0016c001f139a1cb. The SX1302 startup shows the expected AS923 923.2-924.6 MHz multi-SF channel set. The inactive SX1301 placeholder EUI must not be used. UDP Forwarder is disabled.

MQTT Forwarder 4.6.0 already uses the correct local-buffer topology: tcp://127.0.0.1:1883, topic prefix as923, and the active Gateway EUI. However, its current UCI value is **QoS 0**, while the commissioned persistent-buffer design requires QoS 1; treat that as a later bounded configuration correction after the LTE driver path is understood. Do not bypass local Mosquitto.

Local Mosquitto is already running and listening only on 127.0.0.1:1883. Persistence is enabled at /etc/mosquitto/data/mosquitto.db, autosave is 60 seconds, the finite queue limits are 100000 messages and 104857600 bytes, and the local listener is loopback-only. The two existing mTLS bridges use the gateway-specific certificate paths and split uplink/state (out 1, persistent session) from downlink command traffic (in 0, clean session). Their current remote target is lora-test-server:8883; treat that as the existing test endpoint, not as proof that the future public mqtt.<DOMAIN> path is commissioned.

The WaveShare SIM7600G-H is physically detected by the kernel as SimTech USB 1e0e:9001. It appeared first at 1-1.1, disconnected, then re-enumerated at 1-1.3. At the inventory checkpoint there were **no** /dev/ttyUSB* nodes, no /dev/cdc-wdm* node, and no modem-created network interface. The installed packages include generic USB serial/FTDI and PPP support but do not yet prove a matching SIM7600 serial or USB-network driver is present. Do not infer QMI, MBIM, RNDIS, or PPP from the product ID alone; inspect the live USB interface descriptors and bound-driver state first.

The existing OpenWrt logical interface named wwan is **not LTE**: it is the current Wi-Fi station phy0-sta0, address 192.168.8.11/24, with the default route through 192.168.8.1. Keep this management path intact. When the SIM7600 data interface is commissioned, give it a distinct logical name rather than overwriting the working Wi-Fi wwan configuration.

Phase 11.0C USB descriptor result - 2026-08-26

The live 1e0e:9001 device exposes one configuration with six USB interfaces numbered 0 through 5. Every interface is vendor-specific (class ff) and every interface is currently **UNBOUND**. The kernel has generic usbserial plus ftdi_sio and cdc_acm loaded, but the SIMCom option serial driver is not loaded; there are still no /dev/ttyUSB*, /dev/cdc-wdm*, or modem network devices. This rules out treating the current composition as an already-bound RNDIS/CDC-Ethernet/MBIM interface.

Linux's option driver includes the SIMCom/ALINK 1e0e:9001 product family and reserves the higher data/ADB interfaces rather than binding every function as a serial port. For this gateway, commission the **serial option driver first** and verify the resulting ttyUSB layout before choosing the LTE data-plane package. Do not issue new_id, AT+CUSBPIDSWITCH, or another USB-mode change unless the packaged driver fails to recognize the device after installation and a separate diagnostic proves that a manual binding is required.

The first package mutation is therefore split into two gates: refresh the pinned Gateway OS package indexes and verify the matching kmod-usb-serial-option package is available for the running 6.6.141 kernel; only then install that one package and re-check binding. QMI/MBIM/RNDIS packages remain deferred until the post-serial-binding interface state identifies the actual data-plane requirement.

Phase 11.0D package-index result - 2026-08-26

The gateway preserved its Wi-Fi management baseline (phy0-sta0 at 192.168.8.11/24, default route via 192.168.8.1) and confirmed kernel 6.6.141. opkg update completed successfully against the Gateway OS / OpenWrt 24.10.7 feeds with valid signatures. The normal package indexes expose the user-space uqmi and umbim tools, but opkg list returned **zero** kmod-usb-serial-option candidates. No driver was installed and no network or modem USB-mode change occurred.

Do **not** interpret this as unsupported SIM7600 hardware. OpenWrt 24.10 stores kernel-module packages in the target-specific kmods/<kernel-ABI>/ repository, and a normal release image/feed can omit that kmods source from distfeeds.conf; the kmod-usb-serial-option module still exists for Linux 6.6.141. Before editing any feed file, derive the exact installed kernel package/ABI token, current target/subtarget, and existing feed definitions from the live gateway. Add only a kernel-module source whose exact ABI dependency matches the installed kernel; never use --force-depends, a foreign target, or a guessed kernel ABI.

Phase 11.0E exact-kmods result - 2026-08-26

The gateway is ChirpStack Gateway OS 4.12.0 on target bcm27xx/bcm2709, architecture arm_cortex-a7_neon-vfpv4, running kernel 6.6.141. Its installed kernel package is **6.6.141~ce9a7c4f21afbe9986efeaec95ee2cce-r1**. The only official OpenWrt 24.10.7 bcm2709 kmods directory for 6.6.141 is 6.6.141-1-910ca1d362cc3ff4f2a1d9a4e9759bc8; its kmod-usb-serial-option and kmod-usb-serial-wwan packages depend on kernel **6.6.141~910ca1d362cc3ff4f2a1d9a4e9759bc8-r1**. These ABI hashes do not match.

Therefore **do not add that OpenWrt kmods directory as an install source and do not force-install its modules**. Same upstream kernel version does not make separately configured OpenWrt kernel modules ABI-compatible. This matches the known ChirpStack Gateway OS limitation that common USB-4G kmod-* packages may be unavailable for its custom kernel image.

Before committing to a rebuilt Gateway OS image, perform one final read-only capability probe: inspect the running kernel configuration, existing module files, and the already-loaded generic usbserial driver. If the required SIM7600 serial/network functions are not built into this custom kernel and there is no safe existing generic-driver path, the correct durable fix is to build/pin a ChirpStack Gateway OS image from the same source revision with the required USB modem kernel modules included, rather than mixing stock OpenWrt kmods with the custom kernel.

Phase 11.0F current-kernel capability result - 2026-08-26

The read-only kernel probe confirmed the installed Gateway OS image does **not** already contain a usable SIM7600 modem stack. The running kernel remains 6.6.141~ce9a7c4f21afbe9986efeaec95ee2cce-r1. No readable kernel config is exposed through /proc/config.gz or /boot/config-6.6.141, so built-in configuration cannot be inferred from config text; runtime/module evidence is authoritative instead.

The only relevant module file present is /lib/modules/6.6.141/usbserial.ko. Installed packages are kmod-usb-acm, generic kmod-usb-serial, and kmod-usb-serial-ftdi. The USB-serial subsystem exposes only ftdi_sio and the generic driver. There is no packaged/current-kernel option.ko, usb_wwan.ko, qmi_wwan.ko, cdc-wdm.ko, cdc_ether.ko, cdc_ncm.ko, cdc_mbim.ko, or rndis_host.ko, and no SIM7600 module alias database entry is available. All six 1e0e:9001 interfaces remain unbound. Wi-Fi management remains 192.168.8.11/24 with default route via 192.168.8.1.

Although the generic USB-serial driver exposes a writable new_id control, do **not** treat echo 1e0e 9001 > .../generic/new_id as the permanent deployment fix. That broad VID/PID match can claim multiple vendor-specific functions, including interfaces that the SIMCom-aware option driver deliberately excludes, and it does not provide the intended usb_wwan/SIMCom driver behavior. A generic bind may be used only as a separately reviewed temporary diagnostic if later required; it is not the Phase 11 commissioned LTE path.

Current decision: the as-installed Gateway OS 4.12.0 image cannot commission the SIM7600G-H cleanly with its present kernel modules, and the stock OpenWrt kmods cannot be mixed in because of the proven ABI hash

mismatch. Installing drivers directly on the live gateway would be acceptable **only if** an official or locally built package set exists for the exact running Gateway OS kernel ABI. No such matching package set is currently available. Do not use `--force-depends` or copy foreign `.ko` files manually. The supported path is therefore to build or obtain a Gateway OS image/package set containing the required modem drivers compiled together with the Gateway OS kernel, while preserving the existing RAK5146/AS923/gateway-EUI configuration and keeping the current image/config backup available for rollback.

The external build host is needed to **compile an ABI-matched kernel/module set**, not because the SIM7600 must be configured from Windows. The Raspberry Pi gateway remains the runtime target. Building the OpenWrt/Gateway OS kernel on the live gateway is avoided because the gateway has only a small persistent overlay and is already a commissioned radio appliance. The build can use any supported Linux Docker engine; Docker Desktop is optional, not a project requirement.

Upstream ChirpStack Gateway OS supports this directly through its Docker-based source build. The reproducible project procedure is now documented in [Gateway 2A - Build a SIM7600-Capable ChirpStack Gateway OS Image](#). It pins Gateway OS v4.12.0, preserves the existing Base/Raspberry Pi target, builds the SIMCom serial drivers into the same kernel/image, and includes the QMI candidate data path without activating it until runtime proof. A spare card is preferred but not mandatory: with only one SD card, the hard gate becomes a reflash-ready rollback boundary using the exact official factory image plus the already verified off-gateway configuration/private backups. Do not install foreign stock OpenWrt kmods after boot.

Phase 11.0G rollback-backup result - 2026-08-26

PASS on the live gateway; off-gateway copy still required before any build/reflash. The pre-custom-image backup ran without service restarts, network changes, modem changes, or loss of the Wi-Fi management path. The normal Gateway OS configuration archive is `/tmp/gateway-os-backup-20260826-082144.tar.gz`, size about 15.5 KiB, SHA-256 572bfa3f45a69c5ed2ca99263988e8acf2e91ddb514f538520f62d5fb12488a1. It contains the critical Concentratord, MQTT Forwarder, UDP Forwarder, network, and wireless UCI files.

The normal sysupgrade archive contains `/etc/mosquitto/mosquitto.conf` but does **not** contain the Mosquitto certificate tree or persistent data tree. Therefore the explicit protected recovery bundle is required: `/tmp/gateway-critical-private-20260826-082144.tar.gz`, size about 6.0 KiB, SHA-256 f74fc55480bd4edf11a745118e15eae7afac43fff9daabcb64bb20aed4757db1. It was created from the complete `/etc/mosquitto` tree plus the critical UCI files and passed archive-integrity verification. Treat this second archive as secret material because it may contain the gateway MQTT private key and Wi-Fi credentials; never commit it to Git or paste its contents into documentation/chat.

The shell reported hostname: not found; this is a minimal-image utility omission and did not affect the backup gate. Gateway identity remains independently established by the Gateway OS/UBus/Concentratord evidence. **Do not delete the /tmp copies or start the custom-image build until both archives have been copied to protected storage outside the gateway and their SHA-256 values match there.**

The first Windows OpenSSH transfer attempt failed after authentication with `ash: /usr/libexec/sftp-server: not found`. This was a protocol compatibility issue, not a backup-integrity failure: modern OpenSSH `scp` uses SFTP by default, while this minimal Gateway OS image does not provide the SFTP server helper. The retry with `scp -O` forced the legacy SCP protocol and completed successfully.

Phase 11.0G-2 off-gateway preservation result - 2026-08-26

PASS. Both rollback artifacts were copied to protected workstation storage at `C:\Users\smartagriintern\lorawan-recovery\gateway-01\20260826-082144` without changing gateway state. The copied sysupgrade archive is 15908 bytes and its SHA-256 exactly matches 572bfa3f45a69c5ed2ca99263988e8acf2e91ddb514f538520f62d5fb12488a1. The copied protected critical bundle is 6188 bytes and its SHA-256 exactly matches f74fc55480bd4edf11a745118e15eae7afac43fff9daabcb64bb20aed4757db1.

The off-gateway configuration/private rollback boundary is therefore closed: `OFF_GATEWAY_ROLLBACK_COPY=PASS`, `CUSTOM_IMAGE_BUILD_ALLOWED=YES`, `PHASE11_0G2_RESULT=PASS`, and the Windows PowerShell session survived. Because only one SD card is available, do **not** overwrite it yet. Before the first custom-image write, preserve and hash the exact official Gateway OS 4.12.0 Raspberry Pi Base factory image and confirm the workstation has a working SD-card writer. Once that additional gate passes, the same card may be reflashed during a planned maintenance window; rollback then means reflashing the official image and restoring the verified backups, with expected gateway downtime. Retain the gateway /tmp copies until the custom image and restore path have been verified.

Phase 11 build-host consolidation - 2026-08-27

The build-host investigation is complete enough to stop creating micro-gates. The workstation has adequate CPU/RAM/disk, WSL2 is working, Ubuntu 24.04.4 LTS is initialized, Git is present, and Linux networking/DNS is healthy. Keep the Gateway OS source/build tree under /home/smartagriintern/src, not /mnt/c.

The Linux build dependencies are installed and verified inside Ubuntu WSL2: GNU Make 4.3, Docker client/server 29.1.3, `OStype=linux`, `Architecture=x86_64`, and Docker Compose v2 2.40.3+ds1-0ubuntu1~24.04.1. Docker Desktop is **not** a project requirement and is not part of the active path. The build tree is /home/smartagriintern/src/chirpstack-gateway-os.

The intermittent WSL DNS regression was bypassed with a reversible temporary resolver override. Five repeated lookups and `git ls-remote` passed, after which ChirpStack Gateway OS cloned successfully and v4.12.0 was pinned at commit 2112dbdbda48cd77ec1b82499e389abd728e84a1. Pinned submodules are chirpstack-openwrt-feed 2a959fab57cf5a49b843ceac4b0541169e831703 and OpenWrt b40dfac0a31695596f7c1f5f1519302ca8237f6e.

The earlier missing-Compose failure is resolved. The later `make init` stop at `quilt init / No series file found` occurred only after the complete OpenWrt feed update/install and is understood as an initialization-order/tool-version compatibility issue. Do not rerun the full feed installation or invent an empty patch series. The pinned `base_raspberrypi_bcm27xx_bcm2709` switch subsequently passed, `conf/.config`, `conf/files`, and `conf/patches` point to that environment, and all three target patches (`no-uart-console`, `boot-config`, and `image-with-padded-rootfs`) applied successfully.

The next attempt failed before changing modem package state because `openwrt/scripts/config` is a directory in this pinned tree, not a command-line helper. The corrected procedure edits only the eight required `CONFIG_PACKAGE_*` lines, runs `make defconfig`, requires all eight to resolve to `=y`, and then builds with the already-selected target.

Historical handoff note (superseded): at this point on 2026-08-27 the corrected build was still compiling. Later sections prove the build, flash, SIM7600/QMI runtime, LTE path, and the newer 2026-09-01 flash-ready AS923+journal release. Do not use this paragraph as current state.

Parallel rollback preparation result: while compilation continued, the Gateway OS 4.12.0 Base factory artifact was preserved at `C:\Users\smartagriintern\lorawan-recovery\gateway-01\factory-v4.12.0` and hashed as 395e79fe041c4118e10dd4cf796aa426a565d5e733144485d8d014a8d8dbf0a6 (27606919 bytes). Both previously preserved gateway backup archives passed re-hash, balenaEtcher was found at `C:\Users\smartagriintern\AppData\Local\Programs\balena-etcher\balenaEtcher.exe`, and a post-flash checklist was created in the rollback directory. The gateway SD card was not touched. This proves the retained image/hash, backup, imaging-software, and checklist portions of rollback readiness; it does not yet prove a physical SD reader/writer path or maintenance-window acceptance.

From here, execute Phase 11 only from the first unresolved boundary: let the active build finish, verify the custom image/manifest/hash, finish the remaining physical SD reader/writer and maintenance-window rollback checks, then perform the planned reflash and runtime verification. The rollback factory image/hash, backup re-verification, imaging software, and post-flash checklist are already complete. Historical build-host failures remain evidence, not instructions to repeat passed work.

11.1 Goal

Keep two independent outbound paths over the same 4G backhaul:

```
DELIVERY
MQTT Forwarder
-> local Mosquitto persistent queue
-> ssl://mqtt.<DOMAIN>:8883 over 4G

EVIDENCE
Concentrator
-> integrity journal
-> local hash-chained segments
-> https://evidence.<DOMAIN>:443 over 4G
```

MQTT uses a unique gateway certificate. The evidence uploader uses its own per-gateway machine identity when the v2 path is deployed. MQTT Forwarder never connects directly to the cloud and the journal never owns the SPI radio.

11.2 Confirm the gateway and 4G inputs

Before changing the backhaul, confirm the Gateway EUI and legal RF plan are stable, the modem and SIM support the selected carrier/APN, and DNS resolves every enabled machine endpoint.

For delivery confirm:

```
mqtt.<DOMAIN>
MQTT cloud CA
unique MQTT client identity for the Gateway EUI
queue path/finite limits/free-space reserve
```

For v2 evidence also confirm:

```
evidence.<DOMAIN>
evidence server CA
unique evidence-upload identity mapped to the Gateway EUI
journal implementation/version/hash
journal storage budget and latest local chain/checkpoint state
```

Keep the tested Gateway OS image reference, RAK5146 variant, APN reference, endpoints, certificate fingerprints/expiry, and encrypted rollback backup because those values are needed to restore service without changing radio or evidence identity.

Do not copy SIM PINs, APN passwords, or private keys into the manual or test notes.

11.3 Install and configure Gateway OS

Follow:

- [Gateway OS installation](#)
- [Concentrator](#)
- [Persistent MQTT buffer](#)
- [Software-only integrity journal](#)
- [MQTT Forwarder](#)

Keep UDP Forwarder disabled.

11.4 Connect the USB 4G/LTE dongle without losing management access

Keep Ethernet or the currently working management connection attached while commissioning LTE. Do not make LTE the only route until the modem has passed registration, DNS, MQTT, and reconnect tests.

Why: if the APN, modem driver, or route is wrong, keeping the known-good management path prevents a remote lockout while you troubleshoot the gateway.

Before plugging in the dongle, confirm the Raspberry Pi power supply is stable. LTE modems can draw short current bursts, especially during network registration or weak-signal transmit. If the modem repeatedly resets, disappears from USB, or the gateway becomes unstable, investigate power before changing network configuration. A powered USB hub is preferable to guessing at modem settings when power is the actual problem.

The current dongle is the **Waveshare SIM7600G-H 4G DONGLE**. Its documented Linux USB networking options include NDIS/RNDIS/PPP, but the module can expose different USB compositions depending on prior configuration. Treat the **currently enumerated interfaces** as authoritative; do not switch USB PID/mode just because an online example uses a different composition.

After inserting the dongle, inspect what Gateway OS/OpenWrt actually detected:

```
logread | tail -n 100
dmesg | tail -n 100
lsusb 2>/dev/null || true
ip link
ip addr
uci show network
ubus call network.interface dump
```

Look for the SIMCom/Waveshare USB device and the network/control interfaces it created. For this exact dongle, prefer an already-working USB Ethernet/RNDIS-style interface when Gateway OS supports it because it keeps the data path simple; PPP is a fallback only when the live composition and installed release require it. Do not install QMI/MBIM packages unless the device actually exposes the corresponding `cdc-wdm/WWAN` mode.

Common modem presentations include:

```
ECM/RNDIS/HiLink-style modem
-> looks like a USB Ethernet interface
-> normally receives an address with DHCP

QMI modem
-> usually exposes a wwan interface plus a cdc-wdm control device
-> requires the matching QMI support in the pinned OpenWrt build

MBIM modem
-> usually exposes a wwan interface plus a cdc-wdm control device
-> requires the matching MBIM support in the pinned OpenWrt build

Older PPP modem
-> exposes serial ports
-> use only when that exact modem and Gateway OS release are known to require PPP
```

Do not install every modem package blindly. Identify the dongle mode first, then use the package/LuCI procedure supported by the pinned Gateway OS release.

11.4A Configure the carrier/APN

Record the carrier and APN reference outside Git. Configure only the values required by the SIM plan:

```
APN
PDP/IP family when required
SIM PIN only when enabled
username/password only when the carrier actually requires them
```

Do not assume public IPv4. Most mobile connections may sit behind carrier NAT/CGNAT, which is acceptable for this design because the gateway creates outbound TLS connections to the cloud.

No inbound port forwarding to the gateway is required. Do not expose LuCI, SSH, MQTT 1883, or any radio service through the mobile network.

11.4B Verify LTE addressing, routing, DNS, and time

Once the LTE interface reports connected, run:

```
ip addr
ip route
ubus call network.interface dump
ping -c 4 <APPROVED_TEST_HOST>
nslookup mqtt.<DOMAIN>
date -u
```

When gateway evidence is deployed, also verify:

```
nslookup evidence.<DOMAIN>
```

A healthy result shows:

```
LTE/wwan interface has an address
default route exists through the intended LTE interface
route metric is intentional when Ethernet/Wi-Fi also exists
DNS resolves the cloud MQTT hostname
UTC time is correct enough for TLS validation
basic outbound reachability works
```

Interpret failures in this order:

```
No modem/interface
-> USB detection, driver/protocol support, or power problem

Interface exists but has no address
-> SIM registration, APN, PIN, carrier, or modem-session problem

Address exists but no default route
-> interface/network or route-metric configuration problem

IP connectivity works but DNS fails
-> resolver/DHCP/DNS configuration problem

DNS works but MQTT TLS fails
-> cloud firewall, port 8883, certificate, hostname, client identity, or ACL problem
```

Why: this keeps LTE troubleshooting separate from LoRaWAN and MQTT. Do not change the radio region or ChirpStack device configuration to fix a mobile-network routing problem.

11.4C Make LTE the intended cloud path

When Ethernet, Wi-Fi, and LTE coexist, choose route metrics deliberately. During commissioning, keep the management route stable and make sure the cloud-bound route behaves as intended.

Verify again:

```
ip route
nslookup mqttt.<DOMAIN>
```

Then watch the local Mosquitto bridge logs while sending a real uplink. The success condition is not just "the Internet works"; the gateway must establish the MQTT mTLS session over the intended mobile path and ChirpStack must update gateway last-seen.

11.5 Configure the cloud bridge

MQTT Forwarder remains:

```
Server: tcp://127.0.0.1:1883
QoS: 1
Backend: concentratord
```

Local Mosquitto bridge configuration uses:

```
address mqttt.<DOMAIN>:8883
bridge CA: cloud broker CA
bridge certificate: unique gateway certificate
bridge key: matching private key
event/state: out QoS 1
command: in QoS 0, clean session
```

11.6 Network exposure

The gateway initiates outbound TLS. Do not forward internet ports to LuCI, SSH, local MQTT, journal storage, or radio services. Local 1883 must remain on loopback.

Cloud gateway-facing ingress is limited to:

```
TCP 8883 -> MQTT mTLS
TCP 443 -> evidence HTTPS/mTLS, only when the reviewed evidence service exists
```

The evidence API does not expose the journal filesystem and MQTT 8883 does not carry journal segment uploads.

11.7 Normal-path commissioning verification

Before leaving setup, prove the gateway works **without intentionally breaking its WAN or rebooting it**:

```
Gateway EUI stable
RAK5146 / AS923 channel plan correct
Concentratord healthy
UDP Forwarder disabled
MQTT Forwarder -> tcp://127.0.0.1:1883, QoS 1
local Mosquitto listener loopback-only
persistent queue path exists, is writable, finite, and has an OS free-space reserve
```

```
LTE modem registered with intended APN
LTE interface/address/default route/DNS/UTC time healthy
mqtt.<DOMAIN>:8883 reachable through the intended LTE path
unique gateway mTLS certificate accepted
broker ACL permits only that gateway's approved topics
ChirpStack gateway last-seen updates
one real OTAA and uplink succeed
one safe Class A downlink succeeds only when the device has a reviewed command contract
```

For the integrity-journal feature, normal setup additionally requires a **reviewed, pinned implementation** and a healthy current chain/checkpoint path. Record implementation version/hash, current sequence, record/segment hash, storage budget, and server checkpoint. If the selected full-feature scope requires this feature but the reviewed implementation/server roles do not exist, carry that as a Phase 14B BLOCKED item; do not invent a script to make setup look complete.

11.8 Failure tests explicitly deferred to Phase 15

Phase 15 owns LTE disconnect/reconnect, persistent queue growth/drain, gateway reboot with queued messages, stale-downlink prevention, duplicate handling after reconnect, journal offline growth/reconciliation, checkpoint non-advancement while offline, and failure-period data-usage observations.

Phase 11 passes on a verified healthy path plus a correctly configured persistent buffer and, when implemented, a healthy integrity chain. RF and UDP controls remain unchanged.

Next required checkpoint: **Phase 13A** in [13-backup-restore-and-disaster-recovery.md](#), then [12-gateway-and-device-migration.md](#).

Phase 11 current continuation after custom-image cutover - 2026-08-28

The custom Gateway OS image has now booted successfully with SIM7600 support. Runtime proof shows 1e0e:9001, option, usb_wwan, qmi_wwan, cdc_wdm, /dev/ttyUSB0.4, /dev/cdc-wdm0, and modem interface wwan0. The verified Gateway OS configuration was restored, the Mosquitto packages were reinstalled after the factory flash, the protected /etc/mosquitto tree was restored, and the local broker again listens only on 127.0.0.1:1883. MQTT Forwarder reconnects locally as Gateway EUI 0016c001f139a1cb. The old lora-test-server:8883 bridge is retired from the active production path.

Do **not** wait idly for the public domain before continuing. The next provider-independent server step is to finish the cloud gateway MQTT authentication boundary: the current cloud :8884 listener is server-TLS only (require_certificate=false) and no cloud-issued gateway client certificate is recorded. Issue the cloud clientAuth identity for EUI 0016c001f139a1cb, install the exact as923 EUI ACL, harden :8884 to client-certificate authentication canary-first on both brokers, and prove it through the existing anchor :8883 paths using mqtt.internal.lorawan.com. Public FQDN/Reserved-IP/DNS activation follows afterward.

Gateway MQTT cloud client certificate issuance PASS - 2026-08-28

The real physical Gateway EUI 0016c001f139a1cb now has a dedicated cloud MQTT client identity issued on u1c-03 from the commissioned internal CA. Issuance directory: /root/lorawan-pg-ca/gateway-0016c001f139a1cb-issuance-20260828T015149Z; protected transfer bundle is its transfer/ subdirectory. The certificate subject is CN = 0016c001f139a1cb, issuer CN = LoRaWAN PostgreSQL Internal CA, serial D8732205912F3C3AC56E0A01E1E10583, validity 2026-08-28 01:51:51Z through 2027-09-29 01:51:51Z, SHA-256 fingerprint 82:C6:9A:D7:12:5D:8C:45:F3:8F:BA:AB:F9:6E:7E:3B:41:F4:BE:78:95:FE:16:05:FD:58:2E:09:9D:F3:0A:ED, and certificate SHA-256 f348cef6e280dff82722ff908cc96e694faf183eec1dffc55a7abc690cb472d8.

The certificate chains successfully for sslclient, is rejected for sslserver, and its public key matches the RSA-3072 private key. The issuing CA SHA-256 remains 6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb; the existing CA serial-file SHA-256

remained byte-identical at 50df8c462ef9465ab9198284fa1234f0cbfa4f33eb9779ce6d50dd23a618463d because issuance used an explicit random serial. GATEWAY_MQTT_CLIENT_CERT_ISSUANCE=PASS. No Mosquitto, HAProxy, DNS, firewall, or gateway runtime was changed by issuance.

Next boundary: canary-harden ulc-01 gateway-facing Mosquitto :8884 only to require client certificates, map certificate CN to MQTT username, and enforce an exact-EUI as923 gateway ACL. Preserve ChirpStack :8885 and Node-RED :8886. After ulc-01 passes direct mTLS verification with the new gateway certificate, repeat the same bounded rollout on ulc-02.

ulc-01 gateway mTLS canary PASS - 2026-08-28

ulc-01:8884 was hardened from server-TLS-only to gateway client-certificate authentication without changing the ChirpStack :8885 listener. The active gateway listener now has `require_certificate true`, `use_identity_as_username true`, `allow_anonymous false`, and `/etc/mosquitto/gateway.acl`. The exact Gateway EUI 0016c001f139a1cb is permitted to write only its own as923/.../event/# and state/# topics and read only its own command/# hierarchy. Mosquitto restarted successfully, :8884 and :8885 remained listening, the issued gateway certificate completed TLS and MQTT CONNECT/SUBSCRIBE successfully, and a no-client-certificate connection was rejected. Rollback copy: `/etc/mosquitto/gateway-mtls-20260828T015613Z`. Next boundary: apply the same bounded change to ulc-02 and prove the same certificate/ACL behavior there before transferring the cloud certificate bundle to Gateway OS.

ulc-02 gateway mTLS rollout PASS - 2026-08-28

ulc-02:8884 now matches the proven ulc-01 gateway-authentication boundary: `require_certificate true`, `use_identity_as_username true`, `allow_anonymous false`, and `/etc/mosquitto/gateway.acl`. Gateway EUI 0016c001f139a1cb is limited to its own AS923 event/state/command hierarchy. Mosquitto restarted successfully; :8884 and ChirpStack :8885 remained listening. The issued gateway certificate completed TLS, MQTT CONNECT, and own-command SUBSCRIBE successfully, while a no-client-certificate connection was rejected. Rollback copy: `/etc/mosquitto/gateway-mtls-20260828T015801Z`. Therefore both cloud MQTT broker backends now enforce the intended per-gateway mTLS identity. Next boundary: transfer only the three-file cloud certificate bundle (`ca.crt`, `0016c001f139a1cb.crt`, `0016c001f139a1cb.key`) to Gateway OS; do not change the working local Mosquitto topology yet.

ulc-02 gateway mTLS state observed - 2026-08-28

Live read-only inspection after the first rollout wrapper showed ulc-02 is already in the intended gateway mTLS state: `/etc/mosquitto/conf.d/tls.conf` has `listener 8884`, `require_certificate true`, `allow_anonymous false`, `use_identity_as_username true`, and `acl_file /etc/mosquitto/gateway.acl`; `/etc/mosquitto/gateway.acl` exists as 0640 root:mosquitto; `per_listener_settings true` remains active; and the dedicated ChirpStack listener remains 10.104.0.4:8885 with its own password/ACL files. Therefore do not reapply the mutation. The remaining boundary is a direct positive mTLS/MQTT proof with gateway certificate CN 0016c001f139a1cb plus a no-client-certificate rejection proof.

Both cloud gateway MQTT brokers mTLS acceptance PASS - 2026-08-28

Direct verification from ulc-03 against ulc-02:8884 using the issued gateway identity CN = 0016c001f139a1cb passed TLS, MQTT CONNECT, and subscription to the gateway's own as923/gateway/0016c001f139a1cb/command/# hierarchy. A client without a certificate was rejected. Combined with the earlier ulc-01 canary proof, both gateway-facing Mosquitto backends now enforce per-gateway mTLS and the exact EUI ACL while preserving the dedicated ChirpStack :8885 listeners. The cloud broker-side gateway authentication boundary is therefore complete. Next: transfer only `ca.crt`, `0016c001f139a1cb.crt`, and `0016c001f139a1cb.key` from the protected ulc-03 issuance bundle to Gateway OS; do not alter the local loopback broker topology during certificate import.

Gateway cloud MQTT bundle staging PASS - 2026-08-28

The cloud gateway transfer bundle for EUI 0016c001f139a1cb was staged under /home/opsadmin/gateway-mqtt-cloud-0016c001f139a1cb on ulc-03 for protected workstation transfer. The directory contains only ca.crt, 0016c001f139a1cb.crt, 0016c001f139a1cb.key, and SHA256SUMS. Recorded SHA-256 values are CA 6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb, client certificate f348cef6e280dff82722ff908cc96e694faf183eec1dfffc55a7abc690cb472d8, and client key 51726e070cd2b3cdae8e7718650f11673989bd00da80641a6695556ad8b8504d. The private key is mode 0600; staging completed with ULC03_GATEWAY_BUNDLE_STAGING=PASS. Next boundary is workstation copy, hash verification, then legacy-SCP transfer into /tmp on Gateway OS before any certificate installation or Mosquitto restart.

Gateway cloud-certificate workstation relay access-path stop - 2026-08-28

The first cloud-certificate relay attempt did not modify Gateway OS. Windows could not authenticate as opsadmin@159.223.50.57 because the workstation does not have the authorized opsadmin SSH identity, so the 4.5-KiB archive never reached the workstation. Subsequent Windows-to-gateway copy/install commands therefore operated on a nonexistent local file and the gateway-side script stopped at its initial test -s/missing-archive gate. Treat later manually-entered ...=PASS echo lines as non-authoritative; Windows PowerShell 5.1 continued after native-command failures/exceptions. The authoritative archive remains /home/opsadmin/gateway-0016c001f139a1cb-cloud-mqtt-certs.tar.gz on ulc-03, SHA-256 294accecd6080c736c26e3996daba55d19ce62ce0eab52e6d39eea97d698702. Use the already-proven workstation jervis SSH identity for the cloud-to-workstation hop via a temporary 0600 jervis-owned relay copy, then delete that relay copy after workstation hash verification. Do not weaken SSH policy or create/copy an opsadmin private key onto the workstation/server solely for this transfer.

Gateway cloud MQTT certificate transfer PASS - 2026-08-28

The three-file cloud MQTT identity for Gateway EUI 0016c001f139a1cb completed end-to-end protected transfer from ulc-03 to the Windows administration workstation and then to Gateway OS /tmp. The working workstation SSH identity was id_ed25519_home_ops; the first default-agent attempt was correctly rejected. SHA-256 verification passed at the ulc-03 source, on Windows, and again on the gateway for ca.crt (6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb), 0016c001f139a1cb.crt (f348cef6e280dff82722ff908cc96e694faf183eec1dfffc55a7abc690cb472d8), and 0016c001f139a1cb.key (51726e070cd2b3cdae8e7718650f11673989bd00da80641a6695556ad8b8504d). CLOUD_MQTT_CERT_TRANSFER=PASS. No active Gateway OS certificate file, Mosquitto configuration, bridge target, or service was changed by this transfer. Next boundary: install the verified /tmp files into a new off-path /etc/mosquitto/certs candidate, verify chain/CN/key match, preserve the old lab certificate directory as rollback material, then atomically swap the verified candidate into place without restarting Mosquitto or repointing the bridge yet.

Gateway cloud MQTT certificate install runtime-check stop - 2026-08-28

Gateway OS successfully verified the transferred cloud CA, gateway certificate, and private key by SHA-256, built the new certificate directory off-path, verified the sslclient chain, exact Gateway-EUI CN, and certificate/private-key public-key match, preserved the previous active certificate directory at /etc/mosquitto/certs.before-cloud-20260828T02221Z, and activated the cloud files under /etc/mosquitto/certs. The block then stopped immediately after listing the installed files, before the listener check and /tmp cleanup, because the next pidof mosquitto gate returned non-zero (or the utility was unavailable). Treat this as a runtime-verifier stop after successful certificate activation, not as certificate-install failure. Do not roll back the cloud files yet. Next perform one compact Gateway OS check using ps, the OpenWrt init script, and ss/netstat; if Mosquitto is not running, restart it once and require the loopback 127.0.0.1:1883 listener before deleting the retained /tmp transfer copies. Do not change the bridge endpoint in the same step.

Gateway cloud MQTT certificate install PASS - 2026-08-28

The cloud CA and gateway client identity are now installed under `/etc/mosquitto/certs/` on the physical Gateway OS. The active files match the recorded SHA-256 values: `ca.crt`

`6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb, 0016c001f139a1cb.crt`

`f348cef6e280dff82722ff908cc96e694faf183eec1dfffc55a7abc690cb472d8, and 0016c001f139a1cb.key`

`51726e070cd2b3cdae8e7718650f11673989bd00da80641a6695556ad8b8504d`. The certificate chained for `sslclient`, its CN matched `0016c001f139a1cb`, and certificate/private-key public keys matched. The previous certificate directory is preserved at `/etc/mosquitto/certs.before-cloud-20260828T022221Z`.

Post-install runtime verification passed without changing the bridge target: Mosquitto remained running with the local listener on `127.0.0.1:1883`, ChirpStack MQTT Forwarder remained running, and the temporary `/tmp` certificate copies were removed. `GATEWAY_CLOUD_MQTT_RUNTIME_CHECK_EXIT=0`. Repeated TLS errors against `loratest-server:8883` are expected because the bridge still targets the retired lab endpoint while now presenting the cloud client identity; do not treat those errors as a failure of the local persistent-buffer path and do not weaken TLS. The public MQTT FQDN/Reserved-IP/DNS path is still not commissioned, so leave the bridge target unchanged until that endpoint is ready.

Next useful Phase 11 boundary: continue SIM7600/LTE commissioning while preserving the working Wi-Fi management interface `wwan`. Use logical interface name `lte` for the QMI data session on `/dev/cdc-wdm0`; do not overwrite the Wi-Fi `wwan` UCI interface.

SIM7600 AT probe Wi-Fi transient observed - 2026-08-28

After the first `uqmi --get-pin-status` query hung, the operator interrupted it and ran a read-only serial probe against `/dev/ttyUSB2` and `/dev/ttyUSB3` using only AT, ATI, AT+CPIN?, AT+CSQ, and AT+CEREG?. Neither serial port produced a captured AT response. During/after that probe the operator observed a transient disconnect from the Wi-Fi network shared with the gateway. The probe contained no UCI, route, wireless, USB-mode, APN, or data-session mutation, so do not infer that Wi-Fi configuration changed. Do not repeat the AT or QMI probe until read-only gateway logs are checked for Wi-Fi disassociation, USB reset/re-enumeration, and Raspberry Pi undervoltage/power evidence. Preserve the Wi-Fi logical interface `wwan` and do not send `AT+CUSBPIDSWITCH`.

Gateway Wi-Fi DHCP leak confirmed - 2026-08-28

Read-only UCI/runtime inspection proved that management Wi-Fi `wwan` is correctly configured as static `192.168.8.11/24` on `phy0-sta0`, but an unintended `dhcp.wwan` server is also active. The generated `dnsmasq` configuration contains `dhcp-range=set:wwan,192.168.8.100,192.168.8.249,255.255.255.0,12h`, and live logs show the gateway issuing DHCP ACKs to other clients on `lorawan5`. This can conflict with the real upstream DHCP server and explains the observed client connectivity disturbance. The intended project design uses `wwan` only as a management Wi-Fi client and allows SSH/LuCI by placing it in the LAN firewall zone; it does not require DHCP service on `wwan`. First repair only the DHCP leak (`dhcp.wwan.ignore=1`) and verify the generated `wwan` DHCP range disappears before changing firewall-zone duplication, default-route state, or any LTE/modem configuration.

Streamlined management-path decision - 2026-08-28

The operator confirmed that the workstation connectivity problem stopped after unplugging the Raspberry Pi Ethernet cable. Live evidence showed Ethernet `br-lan` and Wi-Fi `phy0-sta0` simultaneously on the same `192.168.8.0/24` network (`192.168.8.131` and `192.168.8.11` respectively), with the default route previously preferring `br-lan`. Treat same-subnet Ethernet/Wi-Fi multi-homing as the primary immediate cause of the observed connection instability. For Phase 11 commissioning, keep Ethernet unplugged and use Wi-Fi `wwan` as the management path. Do not spend time tuning route metrics or firewall duplication unless the problem recurs. Separately, the confirmed `dhcp.wwan` server leak remains a real configuration defect and should be disabled once; then resume LTE commissioning directly.

Management Wi-Fi DHCP leak disabled - 2026-08-28

The gateway management Wi-Fi DHCP leak was corrected by setting `dhcp.wwan.ignore=1`, committing the DHCP configuration, and restarting `dnsmasq`. The operator reported no command errors. Ethernet remains unplugged during LTE commissioning because simultaneous Ethernet and Wi-Fi on the same `192.168.8.0/24` network previously created route ambiguity. Continue with Wi-Fi `wwan` as the management path and use logical interface `lte` for the SIM7600 data session.

SIM7600 scripted microcom probe inconclusive - 2026-08-28

The bounded `microcom` probe against `/dev/ttyUSB2` and `/dev/ttyUSB3` returned no captured output on either port. This does not prove the modem lacks an AT port because the repository has no proven `ttyUSB`-to-function mapping and scripted `stdin/stdout` capture through `microcom` is not authoritative for this Gateway OS. Do not change USB composition. Next use one bounded reader-first AT probe across `/dev/ttyUSB0` through `/dev/ttyUSB4` and stop as soon as a port returns OK.

SIM7600 all-port probe tool limitation - 2026-08-28

The attempted bounded AT probe across `/dev/ttyUSB0..4` was inconclusive because Gateway OS does not provide the `timeout` utility: each background reader exited with status 127 before `cat` could read any serial response. Therefore the empty port outputs are not evidence that the SIM7600 ignored AT commands. Continue with a pure BusyBox/ash background-reader probe that starts `cat`, writes AT, sleeps briefly, then kills only that reader process; do not alter modem USB mode or network configuration.

SIM7600 AT-capable ports proven - 2026-08-28

A corrected BusyBox-only serial read proved that `/dev/ttyUSB2` and `/dev/ttyUSB3` both return live SIM7600 AT responses. The modem reported `+CPIN: READY`, followed by `SMS DONE / PB DONE`, unsolicited SMS indications, and repeated `AT / OK` responses. Treat the SIM as present and PIN-unlocked. Stop scanning all `ttyUSB` ports; use one proven AT-capable port (prefer `/dev/ttyUSB2` for the next bounded status query) and do not send USB-mode-changing commands. The large repeated `AT / OK` stream is probe noise/stale serial traffic and is not a reason to continue port discovery.

SIM7600 LTE registration PASS - 2026-08-28

AT status captured from `/dev/ttyUSB2` proved the SIM7600 and SIM are operational: `+CPIN: READY`, `+CSQ: 22,99`, `+CEREG: 0,1`, and `+COPS: 0,0,"515 66 DITO",7`. Treat SIM unlock, LTE registration on the home network, and usable radio signal as PASS. The repeated standalone `ERROR` lines came from the crude concurrent serial capture and do not invalidate the successful command-specific responses.

SIM7600 PDP-context discovery PASS - 2026-08-28

`AT+CGDCONT?` on `/dev/ttyUSB2` returned three contexts: CID 1 is IP with a blank APN, CID 2 is the carrier IMS context (IPV4V6, APN `ims`), and CID 3 is IPV4V6 with a blank APN. Do not modify CID 2. The normal packet-data context therefore does not currently contain the carrier APN. DITO's published modem/data configuration identifies the APN as `internet.dito.ph`; use that value for the new OpenWrt QMI interface rather than guessing. The custom Gateway OS has already proved `/dev/cdc-wdm0` plus network device `wwan0`, so the next bounded mutation is to create logical UCI interface `lte` with protocol `qmi`, device `/dev/cdc-wdm0`, APN `internet.dito.ph`, and no carrier username/password. Preserve management Wi-Fi logical interface `wwan` unchanged and verify the new UCI stanza before bringing LTE up or changing firewall/default-route behavior.

LTE pre-mutation gate PASS - 2026-08-28

The live gateway has no existing logical `network.lte` interface. Management Wi-Fi remains separate as static logical `wwan` at `192.168.8.11/24` with gateway `192.168.8.1`. QMI runtime devices `/dev/cdc-wdm0` and kernel `wwan0`

exist. Required packages `kmod-usb-net-qmi-wwan`, `uqmi`, and `luci-proto-qmi` are installed. Treat this gate as PASS and do not repeat the driver/package inventory unless later LTE activation fails. The next required input is the modem PDP/APN context; do not invent an APN.

LTE QMI interface configured - 2026-08-28

The live gateway now has logical `network.lte` committed with protocol `qmi`, device `/dev/cdc-wdm0`, APN `internet.dito.ph`, authentication `none`, PDP type `IP`, `defaultroute=0`, and `peerdns=0`. Management Wi-Fi logical `wwan` remains unchanged. Treat this configuration gate as PASS. The next bounded action is `ifup lte`, followed by a single runtime check for interface state and assigned cellular address. Do not repeat driver, SIM, registration, APN, or package gates unless LTE activation fails.

LTE QMI data session PASS - 2026-08-28

`ifup lte` established the DITO QMI packet-data session successfully. Parent logical interface `lte` is up with QMI session identifiers `cid_4` and `pdh_4`; dynamic child interface `lte_4` is up on kernel device `wwan0` and received IPv4 `100.73.25.125/30` with carrier gateway/DHCP server `100.73.25.126`. Carrier DNS advertised `131.226.72.19` and `131.226.73.19`. The default route and peer DNS are intentionally inactive because commissioning uses `defaultroute=0` and `peerdns=0`, preserving management-path behavior. Treat QMI session establishment and LTE address assignment as PASS. Next perform one controlled public-IP connectivity test with a temporary host route through `100.73.25.126`; remove that route immediately after the test.

LTE public connectivity PASS - 2026-08-28

A controlled host-route test proved end-to-end public Internet connectivity over the DITO QMI path without changing the gateway default route. A temporary `/32` route for `1.1.1.1` was installed via carrier gateway `100.73.25.126` on `wwan0` with source `100.73.25.125`; `ip route get 1.1.1.1` confirmed that path and `ping -c 4 -w 3 1.1.1.1` returned two replies with exit status 0. The temporary route was then removed. Treat LTE public IP connectivity as PASS. The observed 50% packet loss is noted for later quality monitoring but does not invalidate the connectivity proof. Keep `network.lte.defaultroute=0` and `peerdns=0` for now: the continuation plan requires the real public MQTT endpoint / certificate / DNS path to be activated and validated before LTE becomes the gateway normal/default route.

11.9 Verified SIM7600 / DITO QMI commissioning path - 2026-08-28

This section is the **known-good path that actually succeeded on the physical gateway**. Use it for the next gateway or for recovery after a clean Gateway OS restore. It intentionally omits failed probe variants and keeps only the commands and checkpoints that produced authoritative PASS results.

11.9.1 Preserve one management path during LTE commissioning

The gateway management Wi-Fi is the OpenWrt logical interface `wwan`, not the SIM7600 data interface:

```
Wi-Fi logical interface: wwan
Wi-Fi kernel device:    phy0-sta0
Management IPv4:       192.168.8.11/24
Management gateway:    192.168.8.1
SIM7600 logical interface: lte
SIM7600 kernel device:  wwan0
QMI control device:     /dev/cdc-wdm0
```

During this commissioning run, Ethernet and Wi-Fi were both attached to the same `192.168.8.0/24` network. Ethernet obtained `192.168.8.131` while Wi-Fi remained `192.168.8.11`, and the default route temporarily preferred `br-lan`. Unplugging Ethernet removed the same-subnet multi-homing ambiguity. Keep Ethernet unplugged while

repeating this procedure unless Ethernet and Wi-Fi are intentionally placed on different subnets or explicit route metrics have been designed.

The restored configuration also contained an unintended DHCP server on the upstream Wi-Fi client interface. Disable that once:

```
uci set dhcp.wwan.ignore='1'
uci commit dhcp
/etc/init.d/dnsmasq restart
```

Why: wwan is a Wi-Fi client on the upstream LAN. It must not hand out leases on lorawan5; the upstream router is the DHCP authority.

11.9.2 Confirm the rebuilt Gateway OS exposes the proven QMI path

The custom Gateway OS build already proved this runtime topology:

```
SIM7600 USB ID: 1e0e:9001
serial devices: /dev/ttyUSB0 .. /dev/ttyUSB4
QMI control:    /dev/cdc-wdm0
QMI netdev:     wwan0
loaded path:    option + usb_wwan + qmi_wwan + cdc_wdm
```

The required packages on the successful gateway are:

```
opkg list-installed | grep -E '(uqmi|luci-proto-qmi|kmod-usb-net-qmi-wwan) '
```

Verified installed packages:

```
kmod-usb-net-qmi-wwan - 6.6.141-r1
luci-proto-qmi - 26.148.37199~3cf713a
uqmi - 2025.07.30~7914da43-r2
```

Do not repeat the package/driver inventory during normal recovery if `/dev/cdc-wdm0` and `wwan0` already exist and the above packages remain installed.

11.9.3 Use the proven SIM7600 AT port

A BusyBox-compatible reader-first probe proved both `/dev/ttyUSB2` and `/dev/ttyUSB3` are AT-capable. Use `/dev/ttyUSB2` for the normal status checks.

The successful modem status set was:

```
AT+CPIN?
AT+CSQ
AT+CEREG?
AT+COPS?
```

Verified responses:

```
+CPIN: READY
+CSQ: 22,99
+CEREG: 0,1
+COPS: 0,0,"515 66 DITO",7
```

Interpretation:

- +CPIN: READY = SIM present and not waiting for a PIN.
- +CSQ: 22,99 = usable/strong radio signal during commissioning.
- +CEREG: 0,1 = registered on the home LTE network.
- +COPS: ... "515 66 DITO",7 = registered to DITO using LTE access technology.

Do not send AT+CUSBPIDSWITCH; the current USB composition is already working.

11.9.4 Read the PDP contexts before configuring OpenWrt

Use the proven AT port and query the modem instead of guessing its current data contexts:

```
AT+CGDCONT?
```

Verified result:

```
+CGDCONT: 1,"IP","", "0.0.0.0",0,0,0,0
+CGDCONT: 2,"IPV4V6","ims", "0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0",0,0,0,0
+CGDCONT: 3,"IPV4V6","", "0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0",0,0,0,1
```

CID 2 is the carrier IMS context and must not be repurposed. The normal data context had no APN configured. The DITO packet-data APN used successfully by this gateway is:

```
internet.dito.ph
```

No carrier username or password was required.

11.9.5 Prove the LTE logical name is free and preserve Wi-Fi wwan

Before creating the LTE interface, the successful pre-mutation state was:

```
uci -q show network.lte || echo 'LTE_INTERFACE_ABSENT=YES'
uci show network.wwan
ls -l /dev/cdc-wdm0 /sys/class/net/wwan0
```

Expected important state:

```
LTE_INTERFACE_ABSENT=YES
network.wwan.ipaddr='192.168.8.11'
network.wwan.gateway='192.168.8.1'
/dev/cdc-wdm0 exists
/sys/class/net/wwan0 exists
```

Why: OpenWrt logical wwan is the management Wi-Fi interface, while Linux kernel wwan0 is the modem netdev. The cellular logical interface is therefore named lte to avoid a namespace collision.

11.9.6 Create the QMI LTE interface without changing the default route

The exact configuration that succeeded is:

```
uci set network.lte='interface'
uci set network.lte.proto='qmi'
uci set network.lte.device='/dev/cdc-wdm0'
uci set network.lte.apn='internet.dito.ph'
uci set network.lte.auth='none'
uci set network.lte.pdptype='IP'
uci set network.lte.defaultroute='0'
```

```
uci set network.lte.peerdns='0'
uci commit network
```

Verify before activation:

```
uci show network.lte
```

Known-good stanza:

```
network.lte=interface
network.lte.proto='qmi'
network.lte.device='/dev/cdc-wdm0'
network.lte.apn='internet.dito.ph'
network.lte.auth='none'
network.lte.pdptype='IP'
network.lte.defaultroute='0'
network.lte.peerdns='0'
```

defaultroute=0 and peerdns=0 are deliberate commissioning controls. They allow the LTE packet-data session to come up without replacing the working Wi-Fi management/default path before the real public MQTT endpoint is ready.

11.9.7 Bring up QMI and verify the dynamic IPv4 child interface

Activate LTE:

```
ifup lte
sleep 10
ubus call network.interface.lte status
ubus list 'network.interface.*' | grep lte
ubus call network.interface.lte_4 status
ip -4 addr show dev wwan0
ip -4 route
```

Verified parent state:

```
network.interface.lte: up
l3_device: wwan0
proto: qmi
cid_4: 18
pdh_4: present
```

Verified dynamic child state:

```
network.interface.lte_4: up
proto: dhcp
device: wwan0
IPv4: 100.73.25.125/30
carrier gateway/DHCP server: 100.73.25.126
carrier DNS: 131.226.72.19, 131.226.73.19
```

The carrier default route and peer DNS remain **inactive** at this stage because network.lte.defaultroute=0 and network.lte.peerdns=0 were intentionally set.

11.9.8 Prove real Internet traffic over LTE without replacing Wi-Fi routing

Use one temporary host route so the test destination is forced through DITO while the gateway's normal default route remains untouched:

```

ip route add 1.1.1.1/32 via 100.73.25.126 dev wwan0 src 100.73.25.125
ip route get 1.1.1.1
ping -c 4 -W 3 1.1.1.1
RC=$?
ip route del 1.1.1.1/32 via 100.73.25.126 dev wwan0 2>/dev/null || true
echo "LTE_PUBLIC_PING_EXIT=$RC"
ip route

```

Verified route selection:

```
1.1.1.1 via 100.73.25.126 dev wwan0 src 100.73.25.125
```

Verified public connectivity:

```

2 replies from 1.1.1.1 out of 4 probes
LTE_PUBLIC_PING_EXIT=0

```

The observed 50% packet loss is recorded for later link-quality monitoring, but it does not invalidate the commissioning result: public IPv4 traffic was proven end-to-end through the DITO LTE data path and the temporary route was removed afterward.

11.9.9 Current verified Phase 11 LTE boundary

The following are now authoritative PASS results:

SIM7600 USB/serial/QMI driver path	PASS
SIM present and PIN-unlocked	PASS
DITO LTE registration	PASS
DITO APN identified	PASS
OpenWrt logical lte interface created	PASS
QMI packet-data session established	PASS
LTE child IPv4 lease	PASS
Carrier gateway and DNS learned	PASS
Public IPv4 traffic over LTE	PASS
Wi-Fi management path preserved	PASS
Unintended Wi-Fi DHCP service disabled	PASS

Current staged topology:

```

Gateway applications / Mosquitto bridge
      |
      | normal route NOT switched yet
      v
Wi-Fi management: wwan / phy0-sta0 / 192.168.8.11

SIM7600 QMI staging path:
logical lte
  |
  | /dev/cdc-wdm0
  |
  | qmi_wwan
  |
  | wwan0 = 100.73.25.125/30
  |
  | DITO gateway = 100.73.25.126
  |
  | Public Internet = proven

```

Do **not** change `network.lte.defaultroute` to 1 yet. The remaining cutover blocker is the production public MQTT path:

```
mqtt.<REAL-DOMAIN>:8883
-> Reserved Public IPv4
-> HAProxy public ingress
-> Mosquitto HA gateway listener
```

First commission the real public FQDN, Reserved IPv4, firewall 8883/tcp, DNS, and broker certificate SAN for that hostname. Then validate the gateway bridge against the real endpoint with mTLS. Only after that proof should LTE become the gateway's intended normal/default route.

2026-09-02 live SIM7600 deep-diagnosis update

Gateway-01 was re-tested after physical unplug/replug and after moving the Waveshare SIM7600G-H from USB path 1-1.1 to 1-1.4. The modem still intermittently emitted Linux USB `qmi_wwan ... Unexpected error -71 (EPROTO)` and at times disconnected/re-enumerated, so USB transport stability remains a real secondary concern. No Pi-wide undervoltage was observed (`vcgencmd get_throttled` returned `0x0`), USB runtime power remained active, and no custom watchdog/hotplug script was found that deliberately resets the modem.

A command-by-command QMI isolation proved that `stop-network`, online mode, 802.3/raw-IP negotiation, WDA query, sync, and network registration did not themselves create new -71 errors. Profile 1 remained `internet.dito.ph`, authentication none. Starting profile 1 with `autoconnect` produced a stable connected bearer and current settings `100.70.234.210/30` via gateway `100.70.234.209` with DITO DNS `131.226.72.19 / 131.226.73.19`.

With a temporary host route through `wwan0`, TCP/HTTP to `1.1.1.1` produced real RX traffic and was redirected by DITO to `filter.dito.ph/nc/`, which returned HTTP 410. This proves the QMI bearer and LTE user plane can carry packets. Standard TLS on port 443 and the production MQTT TLS endpoint `129.212.208.168:8883` did not complete while forced through the LTE bearer. Therefore do not classify the current production-backhaul blocker as an APN, route, QMI-driver, or SIM-registration failure. The strongest current evidence is a DITO restricted / filtered data-service state, with intermittent USB `EPROTO` as a separate secondary reliability issue. Confirm the SIM has an active unrestricted data entitlement/plan before further gateway-side changes. Wi-Fi remains the production default route and Ethernet remains the management path; LTE must not be promoted until unrestricted TLS to the real broker succeeds.

2026-09-02 LTE-primary automatic failover policy

Gateway-01 now uses health-based LTE preference instead of unconditional route metrics. Management Wi-Fi `wwan` has metric 50. Logical `lte` remains QMI on `/dev/cdc-wdm0` with `defaultroute=0`; production now uses `peerdns=1` so DITO's advertised DNS remains usable whenever LTE is the only Internet backhaul. `lte-route-health` runs under `procd` every 30 seconds. Current direct-QMI operation reads the carrier IPv4/gateway from the logical `lte` interface plus QMI current settings and owns the metric-10 `wwan0` default itself. General LTE dataplane health and production MQTT mTLS health are separate signals: a broker/TLS failure does not remove a valid LTE default or reset the modem, because DNS/NTP and recovery must remain possible. Recovery is driven only by repeated loss of the LTE IP dataplane. Ethernet `br-lan 192.168.20.11/24` remains management/recovery only. The stock `mwan3` package was deliberately not forced because its kernel-module dependencies do not match the custom Gateway OS kernel ABI.

2026-09-03 real mobile-data LTE-primary acceptance - PASS

After mobile data became active on the inserted SIM, Gateway-01 was inspected live over the permanent Ethernet management path without changing RF or cloud credentials. The SIM7600 enumerated with `/dev/cdc-wdm0` and `/dev/ttyUSB0.4`; QMI parent `lte` and dynamic child `lte_4` were both up on `wwan0`. The current carrier lease was `100.78.251.124/29` with carrier gateway `100.78.251.125`. `uqmi --get-serving-system` reported `registration=registered`, LTE, MCC 515, MNC 66, non-roaming. Signal at acceptance was RSSI -70 dBm, RSRP -102 dBm, RSRQ -16 dB, SNR -4 dB: usable but worth monitoring rather than treating as strong RF margin.

The health controller automatically installed the intended LTE default:

```
default via 100.78.251.125 dev wwan0 src 100.78.251.124 metric 10
default via 192.168.8.1 dev phy0-sta0 src 192.168.8.132 metric 50
192.168.20.0/24 dev br-lan src 192.168.20.11
```

The production cloud route resolved explicitly as:

```
129.212.208.168 via 100.78.251.125 dev wwan0 src 100.78.251.124
```

This was not only a route-table result. Two live TCP sessions to production MQTT 129.212.208.168:8883 were ESTABLISHED with local source 100.78.251.124, proving the gateway's cloud MQTT path was actually using LTE. A separate HTTPS request explicitly bound to wwan0 reached <https://smartagri-chirpstack.duckdns.org/> at 129.212.208.168, returned HTTP 200, used local IP 100.78.251.124, and reported TLS verification result 0. Thus unrestricted TLS traffic to the real production server is now proven through DITO mobile data, superseding the earlier no-load/restricted-data observation.

Persistence is also present: /etc/rc.d/S95lte-route-health enables the health controller at boot; network.lte remains QMI on /dev/cdc-wdm0, APN internet.dito.ph, defaultroute=0, and now peerdns=1; Wi-Fi remains metric 50. The controller owns the metric-10 LTE default dynamically. It retains that route while the QMI session and general LTE IP dataplane remain usable, even when the production-MQTT probe fails. Enabling LTE peer DNS is necessary because the Mosquitto bridges use the public hostname. In the commissioned field topology, **LTE is the normal production cloud path and Ethernet is management-only**; no automatic Wi-Fi Internet fallback is assumed unless it is deliberately commissioned separately.

2026-09-04 bounded SIM7600 QMI boot-stall recovery - PASS

A later gateway boot exposed a recoverable modem-control stall. /dev/cdc-wdm0 and /dev/ttyUSB0..4 existed and AT returned OK with +CPIN: READY, but network.interface.lte remained pending, wwan0 stayed down, and qmi.sh / uqmi workers were stuck while the kernel had logged early qmi_wwan ... Unexpected error -71. This is not an APN, SIM-presence, or driver-enumeration failure.

Use this recovery only when that exact condition is present and Ethernet management is available:

```
ifdown lte
sleep 2

# Clear only stale QMI setup workers after the logical interface is down.
for p in $(ps w | awk '/qmi\.sh|uqmi/ && !/awk/ {print $1}'); do
    kill "$p" 2>/dev/null || true
done
sleep 1
for p in $(ps w | awk '/qmi\.sh|uqmi/ && !/awk/ {print $1}'); do
    kill -9 "$p" 2>/dev/null || true
done

# Reset only the SIM7600, not the gateway.
stty -F /dev/ttyUSB2 115200 raw -echo 2>/dev/null || true
exec 3<>/dev/ttyUSB2
printf 'AT+CFUN=1,1\r' >&3
exec 3>&-
exec 3<&-

# Wait for /dev/cdc-wdm0 and /dev/ttyUSB2 to re-enumerate, then rebuild QMI.
sleep 10
ifup lte
```

Do not run the kill/reset sequence merely because mobile data is weak or the health probe rejects a restricted SIM. After recovery, require all of these before calling LTE usable: network.interface.lte up=true, dynamic lte_4 IPv4 on wwan0, uqmi --get-serving-system registered on the intended carrier, uqmi --get-data-status connected, ip

route get 129.212.208.168 selecting wwan0, a route-health production MQTT mTLS PASS, and the two Mosquitto bridge sockets established to 129.212.208.168:8883.

The 2026-09-04 recovery produced DITO lease 100.82.66.20/29 via 100.82.66.21, registration MCC/MNC 515/66, signal RSSI -69, RSRP -98, RSRQ -13, SNR 4.2, HTTPS 200 with TLS verification 0 when explicitly bound to wwan0, and two established production MQTT/TLS sockets sourced from 100.82.66.20. The same check exposed the peerdns=0 production defect described above; after changing it to peerdns=1 and restarting only Mosquitto, the hostname-based bridges recovered.

2026-09-16 autonomous stale-session and SIM7600 recovery - PASS

A live research-preflight rehearsal exposed the failure mode the earlier controller did not repair: QMI could report connected, retain an IPv4 address/gateway, and keep a metric-10 route while real Internet, DNS, and MQTT traffic were dead. The controller now proves a general LTE IP dataplane separately from the production broker. Two consecutive dataplane failures trigger bounded stage-1 recovery (ifdown lte / ifup lte). If the dataplane is still unavailable within the escalation window, stage 2 resets only the SIM7600 by stopping stale uqmi, sending AT+CFUN=1,1 to /dev/ttyUSB2, waiting for /dev/cdc-wdm0 and /dev/ttyUSB2 to disappear and re-enumerate, then rebuilding lte. Interface recovery has a cooldown and firmware reset has a 15-minute cooldown to prevent loops. A healthy IP dataplane plus failed broker/TLS probe never triggers either recovery stage.

Both stages were exercised live over Ethernet management. Stage 1 recovered a stale QMI/PDP user plane and restored DNS plus production MQTT mTLS. Stage 2 was then exercised deliberately: the daemon emitted its stage-2 marker, the SIM7600 re-enumerated, QMI returned connected, DITO assigned a fresh 100.86.129.14/30 address via 100.86.129.13, metric-10 wwan0 returned, public IP and DNS passed, the production mTLS health command exited 0, and the daemon failure counter returned to 0. The deployed script SHA-256 matched the repository copy before the reboot-persistence test. A controlled Gateway-01 reboot then proved persistence: /etc/rc.d/S95lte-route-health started the daemon automatically, the repository/deployed SHA-256 remained 39fc435777025484f7543818873d6e106f12fd93e58111a66feace45f6ea995e, early dataplane failures were held inside the 60-second boot grace instead of causing a reset, QMI converged on a fresh DITO 100.69.117.95/26 lease via 100.69.117.96, the metric-10 default returned automatically, and the controller logged LTE dataplane plus production MQTT mTLS PASS by roughly 75 seconds after boot. No manual route, interface, or modem command was needed after reboot.

Supporting runbook: 02a-build-sim7600-capable-gateway-os.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway 2A. Build a SIM7600-Capable ChirpStack Gateway OS Image

Use this manual only when the official ChirpStack Gateway OS Base image does not contain kernel modules required by the WaveShare SIM7600G-H.

The current Phase 11 evidence requires this branch. Gateway OS 4.12.0 on Raspberry Pi 4B runs kernel 6.6.141~ce9a7c4f21afbe9986efeaec95ee2cce-r1, while the stock OpenWrt 24.10.7 kmods repository for the same nominal kernel release requires 6.6.141~910ca1d362cc3ff4f2a1d9a4e9759bc8-r1. These ABI hashes are different. Do not force-install the stock modules into the Gateway OS kernel.

The durable repair is to build the Gateway OS image from its own pinned source/configuration with the modem support selected before the kernel and image are compiled. Direct installation on the live gateway would be acceptable only if a `kmod-*` package set built for the **exact same Gateway OS kernel ABI** were available; the stock OpenWrt packages proven in Phase 11 are not. Do not force those packages or manually copy foreign `.ko` files into `/lib/modules`.

The build host does not have to be Docker Desktop specifically. Any supported Linux environment with a working Docker engine is sufficient for the Gateway OS build. On the current workstation, Ubuntu 24.04 under WSL2 may run its own Linux Docker engine, which is simpler than depending on Docker Desktop WSL integration if Desktop is not already usable.

2A.1 Result

Design invariant: this remains ChirpStack Gateway OS **Base**, not Raspberry Pi OS, Ubuntu, or a generic writable OpenWrt install. Preserve the Gateway OS storage model: an immutable/read-only SquashFS base root filesystem with the normal writable OverlayFS persistence layer on top. The custom build must add only the required modem support and must not convert the image to a conventional fully writable root filesystem, replace the Base profile with Gateway OS Full, or bypass the normal Gateway OS sysupgrade/overlay recovery model.

Build a Raspberry Pi 4B Gateway OS **Base** image that preserves the normal ChirpStack gateway stack and additionally contains:

```
SIM7600 serial/control path
kmod-usb-serial
kmod-usb-serial-wwan
kmod-usb-serial-option

current SIM7600 data-path candidate
kmod-usb-wdm
kmod-usb-net
kmod-usb-net-qmi-wwan
uqmi
luci-proto-qmi
```

```
PPP fallback
keep the existing PPP support selected
```

The QMI packages are included because the current 1e0e:9001 composition exposes a higher vendor-specific data function while the SIMCom-aware option driver is intended for the serial functions. Do not configure QMI as the active WAN until the rebuilt gateway proves /dev/cdc-wdm* plus the associated WWAN interface actually exist.

Do not add Gateway OS Full, a local ChirpStack server, a second packet forwarder, or unrelated packages.

2A.2 Preserve the current working gateway first

Before building or flashing anything, follow [Operations 2. Gateway Backup and Recovery](#).

At minimum preserve outside the gateway:

```
official current Gateway OS 4.12.0 factory-image reference + SHA-256
Gateway OS sysupgrade configuration archive + SHA-256
current Gateway EUI = 0016c001f139a1cb
current radio plan = AS923 / topic prefix as923
current network management configuration
current Mosquitto configuration + encrypted certificate/private-key recovery bundle
current ChirpStack Concentrator/MQTT Forwarder/UDP Forwarder UCI files
```

Phase 11.0G created the current rollback artifacts on the gateway:

```
/tmp/gateway-os-backup-20260826-082144.tar.gz
SHA-256 572bfa3f45a69c5ed2ca99263988e8acf2e91ddb514f538520f62d5fb12488a1

/tmp/gateway-critical-private-20260826-082144.tar.gz
SHA-256 f74fc55480bd4edf11a745118e15eae7afac43fff9daabcd64bb20aed4757db1
```

The normal sysupgrade archive does not cover the Mosquitto certificate/data trees, so preserve **both** artifacts. The second archive is secret-bearing recovery material. Before starting the build, copy both files off the gateway over a protected channel and verify the same SHA-256 values on the receiving machine. Do not commit either archive to this repository.

For this exact Gateway OS image, Windows OpenSSH scp may fail with ash: /usr/libexec/sftp-server: not found because recent OpenSSH clients use SFTP by default and the minimal image does not ship the SFTP server helper. Use legacy SCP mode instead:

```
scp -O root@192.168.8.11:/tmp/gateway-os-backup-20260826-082144.tar.gz root@192.168.8.11:/tmp/gateway-
critical-private-20260826-082144.tar.gz <PROTECTED_DESTINATION>\
```

-O changes only the transfer protocol; it does not mutate gateway configuration.

The Phase 11.0G-2 retry passed. The protected workstation copy is:

```
C:\Users\smartagriintern\lorawan-recovery\gateway-01\20260826-082144
```

Verified destination artifacts:

```
gateway-os-backup-20260826-082144.tar.gz
size 15908 bytes
SHA-256 572bfa3f45a69c5ed2ca99263988e8acf2e91ddb514f538520f62d5fb12488a1

gateway-critical-private-20260826-082144.tar.gz
```

```
size 6188 bytes
SHA-256 f74fc55480bd4edf11a745118e15eae7afac43fff9daabcd64bb20aed4757db1
```

OFF_GATEWAY_ROLLBACK_COPY=PASS and CUSTOM_IMAGE_BUILD_ALLOWED=YES prove the configuration/private recovery artifacts are preserved. A **spare microSD card is preferred but not mandatory**. If no spare card is available, use a controlled single-card reflash path: before overwriting the current card, preserve the exact official Gateway OS 4.12.0 Raspberry Pi Base image outside the gateway, calculate and record its SHA-256, confirm the workstation can write an SD image, keep the verified configuration/private recovery bundles available, and accept planned gateway downtime during rollback. Do not overwrite the only card until those reflash prerequisites pass. Retain the gateway /tmp copies until the custom image has booted and the restore boundary has been verified.

2A.3 Build host requirements

Build on a separate administration/build machine, not on the Raspberry Pi gateway and not on a 2-GiB cloud node used by the POC.

Required:

```
Git
Docker
stable Internet access
substantial free disk space for an OpenWrt source/build tree
```

The upstream Gateway OS build is Docker-based. Building may take a long time and consume many gigabytes; that is normal.

Current build-host state - 2026-08-27

The workstation is already adequate for the build: Windows 10 Pro, 15.6 GiB RAM, 8 logical CPU threads, 126.8 GiB free on C:, WSL2, and Ubuntu 24.04.4 LTS. The Linux account smartagriintern is initialized with sudo, Git 2.43.0 is present, Internet/DNS works, and the Linux filesystem has ample space. Keep the build tree under /home/smartagriintern/src, not /mnt/c.

GNU make and the native Linux Docker toolchain are now installed and proven inside Ubuntu WSL2. GNU Make reports 4.3; Docker client/server report 29.1.3 with OSType=linux and Architecture=x86_64; Docker Compose v2 was subsequently installed and reports 2.40.3+ds1-0ubuntu1~24.04.1. Docker Desktop remains optional and is not used by the active build path.

The build workspace is /home/smartagriintern/src/chirpstack-gateway-os. An intermittent WSL DNS regression was isolated to the WSL-generated resolver 192.168.176.1: direct lookups could pass while git clone failed seconds later. A reversible temporary /etc/resolv.conf using 1.1.1.1 and 8.8.8.8 passed five repeated lookups plus git ls-remote; keep that temporary resolver only while the build may still need source/package downloads.

The repository then cloned successfully and is pinned to Gateway OS tag v4.12.0, commit 2112dbdbda48cd77ec1b82499e389abd728e84a1. Pinned submodules are chirpstack-openwrt-feed at 2a959fab57cf5a49b843ceac4b0541169e831703 and OpenWrt at b40dfac0a31695596f7c1f5f1519302ca8237f6e.

The initial Compose blocker (unknown flag: --rm) is **resolved**, not current. make init then completed the expensive feed update/install and stopped only on the final pre-environment quilt init with No series file found; the later target switch resolved the real patch path and successfully applied the Raspberry Pi target patches. The Raspberry Pi Base environment is therefore already selected. Do not rerun Docker installation, clone, feed installation, or target switching merely to reproduce earlier evidence.

The next attempted non-interactive package edit failed before changing modem package state because openwrt/scripts/config is a directory in this pinned tree. The corrected build block uses minimal .config updates

plus make `defconfig` and an eight-package verification gate. **This is historical troubleshooting, not current status:** later sections prove the build, artifact hashes, physical flash, SIM7600/QMI runtime, LTE connectivity, and the superseding 2026-09-01 flash-ready AS923+journal image. Do not resume from this intermediate point.

The earlier H-1/H-2/H-3/H-4A probes are retained as historical evidence in the Phase 11 deployment log, not as separate operator gates. From this point use one streamlined build flow:

```
Ubuntu WSL2
-> install/verify make + Linux Docker
-> clone Gateway OS v4.12.0 under Linux home
-> select bcm27xx/bcm2709 Base
-> add only SIM7600 serial/QMI packages
-> build
-> verify image/manifest/hash
-> confirm single-card rollback is reflash-ready
-> planned reflash
-> verify RAK5146 + Gateway EUI + AS923 + SIM7600
```

Stop only on a real safety/correctness failure: build dependency unavailable, wrong target/release, required package missing, build/hash verification failure, or rollback prerequisites incomplete. Do not create a new micro-phase merely for a successful read-only command.

2A.4 Pin the Gateway OS source

Clone the official repository on the build host:

```
git clone https://github.com/chirpstack/chirpstack-gateway-os.git
cd chirpstack-gateway-os
git fetch --tags --force
git checkout --detach v4.12.0
git status --short
git describe --tags --exact-match
git rev-parse HEAD
```

Record the full commit returned by `git rev-parse HEAD` with the build evidence. Do not build from `master` merely because it is newer. The running gateway is on Gateway OS 4.12.0, so reproduce that release first and change only the required modem package selection.

2A.5 Initialize the pinned OpenWrt tree

From the Gateway OS repository root:

```
make init
```

This initializes the exact OpenWrt source and feeds used by the pinned Gateway OS source tree.

Current v4.12.0 initialization compatibility note - 2026-08-27

On the current build host, `make init` completed the expensive/required initialization work successfully: both pinned submodules were checked out, all OpenWrt feeds were updated, and all feeds were installed. It then stopped only on its final command:

```
docker compose run --rm chirpstack-gateway-os quilt init
No series file found
make: *** [Makefile:17: init] Error 1
```

Do **not** repeat the feed update/install after this exact failure. The upstream Compose environment sets `QUILT_PATCHES=/workdir/conf/patches`, while the upstream `switch-env` target creates the `conf/patches` symlink only after a target environment is selected. With the current Debian `stable-slim` Quilt package, `quilt init` rejects the missing `series` file before that target link exists. Treat this as an initialization-order/tool-version compatibility issue, not a failed OpenWrt feed initialization.

For this exact state, continue directly with `make switch-env ENV=base_raspberrypi_bcm27xx_bcm2709` inside the build container. Its initial `quilt pop -a` is intentionally non-fatal, then it creates the target `.config`, `files`, and `patches` symlinks and applies the target patch series. Pass only if the target patch series exists and `quilt push -a` completes successfully. Do not create an ad-hoc empty `series`, alter the upstream Makefile, or rerun all feed installation merely to make the final `quilt init` return zero.

Then enter the Docker build environment:

```
make devshell
```

Run the remaining build commands **inside that development shell** unless the upstream build environment explicitly returns you to the host shell.

2A.6 Select the Raspberry Pi 4B Base environment

Inside the development shell:

```
make switch-env ENV=base_raspberrypi_bcm27xx_bcm2709
```

Verify the selected environment before editing package selection:

```
grep -E '^CONFIG_TARGET_bcm27xx|^CONFIG_TARGET_bcm27xx_bcm2709' openwrt/.config 2>/dev/null || \  
grep -E '^CONFIG_TARGET_bcm27xx|^CONFIG_TARGET_bcm27xx_bcm2709' .config
```

Stop if this is not the Raspberry Pi `bcm27xx/bcm2709` Base environment.

2A.7 Add only the required modem packages

The Raspberry Pi Base environment switch has already passed and applied all three target patches. On this pinned OpenWrt tree, `openwrt/scripts/config` is a **directory containing the Kconfig implementation**, not the command-line helper used by some other projects. Do not call `./scripts/config --enable ...`

For an interactive build, the upstream-supported method remains:

```
cd openwrt  
make menuconfig
```

For this reproducible Phase 11 build, use the equivalent non-interactive `.config` update: remove any existing value or not `set` line for only the required package symbols, append `=y`, then let OpenWrt make `defconfig` resolve and validate dependencies. This is the same `.config` input path used by OpenWrt's build system; `defconfig` is the authoritative normalization step.

Required packages:

```
kmod-usb-serial  
kmod-usb-serial-wwan  
kmod-usb-serial-option
```

```
kmod-usb-wdm
kmod-usb-net
kmod-usb-net-qmi-wwan
uqmi
luci-proto-qmi
```

Keep the existing Gateway OS Base selections and existing PPP support. Do not deselect RAK/ChirpStack packages and do not add unrelated modem frameworks merely because they are present in the feeds.

After the eight symbols are changed, run:

```
make defconfig
```

Then prove the intended package selection before compiling:

```
grep -E '^CONFIG_PACKAGE_(kmod-usb-serial|kmod-usb-serial-wwan|kmod-usb-serial-option|kmod-usb-wdm|kmod-usb-net|kmod-usb-net-qmi-wwan|uqmi|luci-proto-qmi)=' .config
```

All eight packages must resolve to `=y`. If `make defconfig` removes or changes any requested symbol, stop and inspect that symbol's dependencies instead of forcing the build.

2A.8 Build the image

From the environment expected by the upstream repository:

```
make
```

Do not interrupt the first build simply because it is slow.

After success, inspect the Raspberry Pi target output. The OpenWrt target artifacts normally live under:

```
openwrt/bin/targets/bcm27xx/bcm2709/
```

Locate the Base factory image and its manifest rather than guessing a filename:

```
find openwrt/bin/targets/bcm27xx/bcm2709 \
  -maxdepth 1 -type f \
  \( -name '*.img.gz' -o -name '*.wic.gz' -o -name '*.manifest' -o -name 'sha256sums' \) \
  -print
```

2A.8A Use compile time for independent rollback/readiness work

While the custom image is compiling, prepare only items that do not mutate the active source tree or the gateway SD card:

1. Download the official ChirpStack Gateway OS 4.12.0 Raspberry Pi 4B **Base factory image** from the ChirpStack Raspberry Pi download page and calculate its SHA-256 locally. Keep both the image and hash outside the gateway as the single-card rollback source.
2. Re-verify the existing off-gateway sysupgrade archive and protected Mosquitto/configuration bundle against their already-recorded SHA-256 values. Do not open or print secret-bearing file contents.
3. Confirm the workstation has an SD-card reader/writer and an imaging application ready, but do **not** remove or overwrite the gateway's only card while the build is running.

4. Record the carrier/APN information for the installed SIM, including whether the SIM requires a PIN. Do not send USB-mode-changing AT commands.
5. Keep a post-flash verification sheet ready for: Gateway OS release/kernel, RAK5146, Gateway EUI 0016c001f139a1cb, AS923, Wi-Fi management, Mosquitto persistence, MQTT Forwarder local endpoint, UDP Forwarder disabled state, SIM7600 USB binding, /dev/ttyUSB*, and QMI/PPP evidence.
6. Keep the temporary WSL DNS override in place until all source/package downloads are finished; restore the original WSL resolver only after the build no longer needs network access.

These preparations may run in parallel with compilation because they do not alter openwrt/.config, build outputs, the running gateway, or its SD card.

Current parallel-preparation result - 2026-08-27

The operator completed the non-destructive rollback preparation while the custom image continued compiling:

```
factory rollback artifact:
C:\Users\smartagriintern\lorawan-recovery\gateway-01\factory-v4.12.0\chirpstack-gateway-os-4.12.0-base-
bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz
size: 27606919 bytes
SHA-256: 395e79fe041c4118e10dd4cf796aa426a565d5e733144485d8d014a8d8dbf0a6

existing sysupgrade backup re-hash: PASS
existing protected configuration/private backup re-hash: PASS

SD imaging software:
C:\Users\smartagriintern\AppData\Local\Programs\balena-etcher\balenaEtcher.exe
SD_WRITER_SOFTWARE=PASS

post-flash checklist:
C:\Users\smartagriintern\lorawan-recovery\gateway-01\factory-v4.12.0\PHASE11-POST-FLASH-CHECKLIST.txt

GATEWAY_SD_CARD_TOUCHED=NO
```

Treat these as completed prerequisites. Do not confuse SD_WRITER_SOFTWARE=PASS with proof that the physical SD reader/writer path has been exercised. Before the only production card is overwritten, still confirm the physical reader/writer path and accept the maintenance-window downtime. Carrier/APN/PIN information also remains a later LTE-commissioning input.

Build-finished status - 2026-08-27

The operator reports that the Phase 11 custom Gateway OS compilation has finished. This does **not** yet close the build branch: the final command result and generated artifacts still require verification. Preserve the existing build tree and inspect it incrementally; do not rerun make, make init, or the target switch unless evidence shows the build actually failed or an artifact is missing.

Before flashing, capture and verify the existing outputs exactly as described in section 2A.9. In particular, require all eight modem packages in the generated manifest and record SHA-256 for the factory image, manifest, and final .config. The production SD card remains untouched.

2A.9 Verify the custom image before flashing

Record SHA-256 for the exact factory image:

```
sha256sum <CUSTOM_GATEWAY_OS_FACTORY_IMAGE>
```

Inspect the generated manifest and require the modem packages:

```
grep -E '^(kmod-usb-serial|kmod-usb-serial-wwan|kmod-usb-serial-option|kmod-usb-wdm|kmod-usb-net|kmod-usb-net-qmi-wwan|uqmi|luci-proto-qmi)' ' \
<CUSTOM_IMAGE_MANIFEST>
```

Also preserve:

```
Gateway OS tag + full source commit
OpenWrt target = bcm27xx/bcm2709
custom .config hash
factory-image filename + SHA-256
manifest filename + SHA-256
build date/time in UTC
```

Do not flash an image whose manifest does not contain the intended modem support.

2A.10 First boot on spare or controlled single-card media

A spare microSD card remains the preferred first-boot path. When only the single production card exists, do not write it until the reflash-ready rollback gate is closed. Current evidence already covers the Gateway OS 4.12.0 Base factory artifact plus local SHA-256, the verified configuration/private recovery bundles, and installed imaging software. The custom image/manifest is now verified and preserved. Still require only a confirmed physical SD reader/writer path and maintenance-window acceptance before writing the card.

Flash the custom **factory** image using the procedure in [Gateway 2](#). On the single-card path, this is the planned maintenance-window cutover and rollback means reflashing that same card with the preserved official factory image before restoring the verified backups.

Before restoring the production-like configuration, boot the custom image and verify only the platform/modem layer:

```
cat /etc/os-release
uname -r
opkg status kernel
opkg list-installed | grep -E '^kmod-usb-(serial-option|serial-wwan|net-qmi-wwan|wdm|net)'
lsmod | grep -E 'option|usb_wwan|qmi_wwan|cdc_wdm|usbserial'
ls -l /dev/ttyUSB* /dev/cdc-wdm* 2>/dev/null
ip -br link 2>/dev/null || ip link
logread | grep -Ei '1e0e|9001|option|ttyUSB|qmi|cdc-wdm|wwan' | tail -n 150
```

Expected direction for the current SIM7600G-H composition:

```
1e0e:9001 detected
serial interfaces bind to option
/dev/ttyUSB* appears
higher data function binds only to the selected data driver when supported
/dev/cdc-wdm* and a WWAN network interface appear if QMI is active
```

Do not change the modem USB PID with AT+CUSBPIDSWITCH merely to make the expected output appear.

2A.11 Restore the gateway configuration only after driver proof

Once the custom image proves the modem drivers load correctly on the selected spare-card or controlled single-card path:

1. restore the protected Gateway OS configuration archive;
2. verify Gateway EUI remains 0016c001f139a1cb;
3. verify Concentratord remains RAK5146 / AS923;

4. keep UDP Forwarder disabled;
5. verify MQTT Forwarder still targets 127.0.0.1:1883;
6. verify Mosquitto remains loopback-only and persistent;
7. preserve the existing Wi-Fi management path during LTE commissioning;
8. name the future cellular logical interface `lte`, not `wwan`, because `wwan` is already the Wi-Fi management interface in this gateway.

If spare media is available, keep the original working microSD card untouched until the rebuilt gateway passes normal-path verification. On the single-card path, do not begin the first write until the reflash-ready rollback gate above is complete; after writing, retain every off-gateway rollback artifact until normal-path verification passes.

2A.12 Pass condition

This image-build branch passes when:

```
source pinned to Gateway OS v4.12.0
Raspberry Pi Base bcm27xx/bcm2709 environment used
custom image manifest contains required SIM7600 serial drivers
stock OpenWrt foreign-ABI kmods were not installed
custom image boots on the selected spare-card or controlled single-card path
RAK5146 remains detected
SIM7600 1e0e:9001 binds to the intended drivers
/dev/ttyUSB* exists
QMI control/network interface exists if QMI is the proven data path
working gateway configuration can be restored without changing EUI/AS923 identity
rollback remains possible through the preserved official factory image plus verified off-gateway
configuration/private backups
```

After this passes, return to [Phase 11](#) for SIM registration, APN, LTE routing, QoS 1 correction, and cloud MQTT commissioning.

Post-build verifier correction - 2026-08-27

Observed verifier-only failure. The first post-build output check was launched from `/mnt/c/Users/smartagriintern`, so `git describe / git rev-parse` failed with `not a git repository` before any build artifact was inspected. This does **not** establish that the Gateway OS build failed. The operator login shell survived and no SD-card write occurred.

A second issue was found in that verifier before retry: it converted package names such as `kmod-usb-serial` to underscores when constructing `CONFIG_PACKAGE_...` symbols. OpenWrt package configuration symbols retain the package-name hyphens (for example `CONFIG_PACKAGE_kmod-usb-serial=y`). The corrected verifier must therefore `cd ~/src/chirpstack-gateway-os` explicitly and test the exact hyphenated symbols.

Do not rebuild. Resume from read-only output verification only. Do not flash until image, manifest, required package membership, and hashes are all proven.

Custom-image Stage 1 verification PASS - 2026-08-27

The corrected read-only post-build verifier completed successfully. The earlier verifier failure was only a wrong-working-directory/script issue and did not invalidate the build.

Verified source and target:

```
Gateway OS tag: v4.12.0
Gateway OS commit: 2112dbdbda48cd77ec1b82499e389abd728e84a1
OpenWrt target: bcm27xx/bcm2709
```

All eight required SIM7600 packages are present in both the final OpenWrt .config and the generated image manifest:

```
kmod-usb-serial
kmod-usb-serial-wwan
kmod-usb-serial-option
kmod-usb-wdm
kmod-usb-net
kmod-usb-net-qmi-wwan
uqmi
luci-proto-qmi
```

Generated factory image:

```
chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz
size: 27672032 bytes
SHA-256: 1540958a7247e78e0e0d6791e3550f5d608865e940fc7463a5ff68229199bc3f
```

Generated manifest:

```
chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2.manifest
size: 8009 bytes
SHA-256: 49ea8900d5edccd89bafb6868f7918b71eb28c15481b67e390d69b6e7d513561
```

Final OpenWrt configuration SHA-256:

```
d17b44d29cbfc54ef40734532ee5ce900c9ff110b95c9f5107f54bbb0753cdfa
```

The target directory also contains a generated sha256sums file. Stage 1 establishes that the intended factory image exists and that its manifest contains the complete required modem package set. Before any SD-card write, still verify the generated sha256sums against the produced artifacts, preserve the custom build evidence outside the WSL build tree, restore the temporary WSL DNS after preservation, confirm the physical SD reader/writer path, and explicitly accept the maintenance-window downtime.

GATEWAY_SD_CARD_TOUCHED=NO remains authoritative.

Custom-image Stage 2 artifact-integrity PASS - 2026-08-28

The read-only artifact-integrity gate completed successfully without rebuilding, changing DNS, or touching the gateway SD card. The previously recorded factory-image, manifest, and final OpenWrt .config hashes were reverified exactly.

Generated sha256sums verification passed for every listed target artifact:

```
chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz: OK
chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-sysupgrade.img.gz: OK
chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2.manifest: OK
config.buildinfo: OK
feeds.buildinfo: OK
profiles.json: OK
version.buildinfo: OK
```

Authoritative hashes remain:

```
factory image SHA-256: 1540958a7247e78e0e0d6791e3550f5d608865e940fc7463a5ff68229199bc3f
manifest SHA-256: 49ea8900d5edccd89bafb6868f7918b71eb28c15481b67e390d69b6e7d513561
final openwrt/.config SHA-256: d17b44d29cbfc54ef40734532ee5ce900c9ff110b95c9f5107f54bbb0753cdfa
sha256sums SHA-256: 73ff93c061d87b9e4231776dcb420382a8e539646b64847b8e10ed4498f67e90
```

Reliable UTC mtimes recorded from filesystem epoch timestamps:

```
factory image: 2026-08-27T05:53:55Z
manifest: 2026-08-27T05:53:56Z
sha256sums: 2026-08-27T05:54:10Z
integrity verification: 2026-08-28T00:46:41Z
```

The factory image and manifest are both explicitly listed in the generated sha256sums. Stage 2 closes the custom-build artifact-integrity gate. Remaining pre-flash work is: preserve the verified custom artifacts and evidence outside WSL, verify the copied hashes, restore the temporary WSL DNS only after preservation, confirm the physical SD reader/writer path, and accept the maintenance-window downtime.

BUILD_REEXECUTED=NO, DNS_CHANGED=NO, and GATEWAY_SD_CARD_TOUCHED=NO remain authoritative for this gate.

Custom-image Stage 3 off-WSL preservation PASS - 2026-08-28

The verified custom Gateway OS output set was copied from the WSL build tree to Windows recovery storage and revalidated there without rebuilding, changing DNS, or touching the production SD card.

Preserved location:

```
C:\Users\smartagriintern\lorawan-recovery\gateway-01\custom-v4.12.0-sim7600-20260827
```

The preserved set includes the factory image, sysupgrade image, manifest, config.buildinfo, feeds.buildinfo, profiles.json, generated sha256sums, version.buildinfo, a copy of the final OpenWrt .config, and a non-secret build-identity record.

Windows-side revalidation passed exactly:

```
factory image SHA-256: 1540958a7247e78e0e0d6791e3550f5d608865e940fc7463a5ff68229199bc3f
manifest SHA-256: 49ea8900d5edccd89bafb6868f7918b71eb28c15481b67e390d69b6e7d513561
final OpenWrt .config SHA-256: d17b44d29cbfc54ef40734532ee5ce900c9ff110b95c9f5107f54bbb0753cdfa
generated sha256sums SHA-256: 73ff93c061d87b9e4231776dcb420382a8e539646b64847b8e10ed4498f67e90
```

Running sha256sum -c sha256sums against the Windows-side preserved target directory also returned OK for every generated artifact listed by OpenWrt. This closes the off-WSL custom-build preservation gate.

The DNS restoration gate is now complete. Remaining pre-flash gates are only: confirm the physical SD reader/writer path and explicitly accept the maintenance-window downtime before writing the only production card.

BUILD_REEXECUTED=NO, DNS_CHANGED=NO, and GATEWAY_SD_CARD_TOUCHED=NO remain authoritative for this preservation gate.

Custom-image Stage 4 WSL DNS restoration PASS - 2026-08-28

The temporary public-DNS workaround is no longer active. /etc/resolv.conf was already restored to the normal WSL-generated symlink /mnt/ws1/resolv.conf, whose current resolver is 192.168.176.1. Five consecutive getent ahosts github.com checks passed, so no DNS rollback was required.

Verified boundary:

```
WSL_GENERATED_RESOLVER_PRESENT=PASS
WSL_DNS_ALREADY_RESTORED=YES
WSL_RESOLV_LINK_RESTORED=ALREADY_PRESENT
WSL_RESOLV_LINK_VERIFIED=PASS
dns_lookup_1=PASS
dns_lookup_2=PASS
dns_lookup_3=PASS
```

```
dns_lookup_4=PASS
dns_lookup_5=PASS
ORIGINAL_WSL_DNS_VALIDATION=PASS
DNS_OVERRIDE_RESTORED=YES
BUILD_REEXECUTED=NO
GATEWAY_SD_CARD_TOUCHED=NO
PHASE11_WSL_DNS_RESTORE=PASS
PHASE11_WSL_DNS_RESTORE_EXIT=0
LOGIN_SHELL_SURVIVED=YES
```

The custom build, artifact integrity, off-WSL preservation, and workstation DNS cleanup gates are now closed. The physical SD reader/removable disk path is now proven. The only remaining prerequisite before shutdown/card removal is explicit maintenance-window/cutover acceptance. Do not flash until that approval is given.

Phase 11 physical SD reader preflight shell-context failure - 2026-08-28

The first physical SD reader preflight did **not** execute because the Windows PowerShell block was pasted into the Ubuntu/WSL Bash shell. Bash rejected PowerShell syntax (& {, Write-Host, Get-CimInstance, Get-PnpDevice) before any Windows disk or PnP inventory was performed. Treat this as a shell-context/verifier failure only, not as an SD-reader hardware failure.

Authoritative safety boundary from the failed attempt:

```
PHYSICAL_SD_READER_PREFLIGHT=NOT_EXECUTED
DISK_WRITE_EXECUTED=NO
DISK_INITIALIZE_EXECUTED=NO
DISK_FORMAT_EXECUTED=NO
IMAGE_WRITE_EXECUTED=NO
GATEWAY_SD_CARD_REMOVED=NO
PRODUCTION_SD_CARD_TOUCHED=NO
```

Retry by invoking Windows PowerShell explicitly from WSL or by running the block in a native Windows PowerShell prompt. Do not remove the production microSD until the empty-reader hardware baseline has passed.

Streamlined Phase 11 cutover convention - 2026-08-28

From this checkpoint onward, avoid repeating already-passed build, checksum, preservation, DNS, imaging-software, and rollback checks unless a later failure gives a concrete reason. The remaining workflow is intentionally simple:

1. confirm Windows sees the SD reader / intended removable disk;
2. accept the maintenance-window downtime;
3. flash the verified custom factory image;
4. boot the gateway and verify Gateway OS, LoRa radio, Wi-Fi management, packet forwarding, and SIM7600 driver presence;
5. continue LTE commissioning only if those checks work.

A working result is sufficient evidence. Do not add extra test layers merely to restate already-proven state.

The PowerShell reader-preflight attempt that was blocked by Windows script execution policy is a launcher issue only, not a hardware failure. `PRODUCTION_SD_CARD_TOUCHED=NO` remains authoritative.

Physical SD reader / removable disk detection PASS - 2026-08-28

Windows detected the connected SD reader/media path clearly enough for cutover:

```
Disk 1
Model: Mass Storage Device USB Device
InterfaceType: USB
```

MediaType: Removable Media
Size: 31946987520 bytes

Treat this as sufficient physical reader/disk evidence for the streamlined Phase 11 cutover. Do not repeat reader diagnostics unless the device later disappears or imaging software cannot see the card.

The production SD card has not yet been removed or written. The only remaining gate before shutdown/card removal is explicit maintenance-window/cutover acceptance. After approval, proceed directly to controlled shutdown, remove the card, re-identify the removable disk by model/size, flash the verified custom factory image, boot, and verify the gateway works.

PRODUCTION_SD_CARD_TOUCHED=NO remains authoritative at this checkpoint.

Phase 11 maintenance-window / cutover approval - 2026-08-28

The operator explicitly approved proceeding with the Phase 11 single-card cutover. Planned gateway downtime and the documented rollback path are accepted.

Current cutover sequence is intentionally streamlined:

1. gracefully power off the live gateway at 192.168.8.11;
2. remove the production microSD only after shutdown completes;
3. insert it into the already-proven Windows removable-media path and confirm the same ~31.9 GB device;
4. flash the verified custom Gateway OS factory image;
5. reinstall the card, boot, and verify LoRa, Wi-Fi management, existing MQTT path, and SIM7600 driver presence.

Do not repeat already-passed build, checksum, preservation, DNS, or reader checks unless a later step actually fails.

MAINTENANCE_WINDOW_ACCEPTED=YES CUTOVER_APPROVED=YES PRODUCTION_SD_CARD_TOUCHED=NO at this approval boundary.

Production microSD identified for cutover - 2026-08-28

Windows re-identified the inserted production microSD immediately before flashing:

Disk 1
Model: Mass Storage Device USB Device
Interface: USB
Media: Removable Media
Size: 31946987520 bytes

This matches the previously observed removable reader/media path and is sufficient for the streamlined cutover. Proceed with balenaEtcher using the preserved custom factory image under C:\Users\smartagriintern\lorawan-recovery\gateway-01\custom-v4.12.0-sim7600-20260827\target-output\. Do not select the same-named official rollback image under factory-v4.12.0.

Production microSD custom-image flash PASS - 2026-08-28

The approved Phase 11 single-card cutover has now written the verified custom ChirpStack Gateway OS factory image to the production microSD using balenaEtcher, and the operator reports the flash completed successfully. The written source was the preserved custom image under C:\Users\smartagriintern\lorawan-recovery\gateway-01\custom-v4.12.0-sim7600-20260827\target-output\, not the same-named official rollback image under factory-v4.12.0.

Because a factory image replaces the previous card contents, do not assume the prior Wi-Fi management address/configuration is already present on first boot. First boot should use Ethernet DHCP when available or the Gateway OS commissioning AP, allow first-boot filesystem expansion/reboot to finish, and prove the SIM7600 runtime driver path before restoring the verified gateway configuration/private backup.

Current cutover state:

```
CUSTOM_FACTORY_IMAGE_FLASHED=YES
BALEANAETCHER_FLASH_REPORTED_SUCCESS=YES
PRODUCTION_SD_CARD_WRITTEN=YES
ROLLBACK_IMAGE_PRESERVED=YES
NEXT=BOOT_CUSTOM_IMAGE_AND_PROVE_MODEM_DRIVERS
```

Custom-image first-boot SIM7600 runtime proof PASS - 2026-08-28

The freshly flashed custom Gateway OS booted successfully and runtime modem binding is proven. This closes the custom kernel/module objective; do not repeat build/package checks unless a later runtime regression appears.

Observed runtime evidence:

```
SIM7600 USB ID: 1e0e:9001
modules loaded: option, usb_wwan, qmi_wwan, cdc_wdm, usbserial
serial devices: /dev/ttyUSB0 through /dev/ttyUSB4
QMI control: /dev/cdc-wdm0
QMI network device: wwan0
qmi_wwan registered the modem data interface successfully
```

The custom image therefore provides both the SIMCom serial path and a working QMI-capable control/network path. The next step is to restore the verified Gateway OS configuration archive, then restore the protected Mosquitto/private bundle as required, and re-verify only the operational identity/path: RAK5146, Gateway EUI 0016c001f139a1cb, AS923, Wi-Fi management, local Mosquitto/MQTT Forwarder path, and UDP Forwarder disabled.

Normal Gateway OS configuration restore PASS - 2026-08-28

The verified gateway-os-backup-20260826-082144.tar.gz was restored successfully after the custom-image SIM7600 driver proof. The gateway returned to its working management/configuration state. Treat this restore step as PASS and do not repeat it unless a later service check fails. Next restore the protected Mosquitto/private bundle, then run one compact identity/service verification.

Mosquitto package reinstall PASS - 2026-08-28

After the custom factory-image flash and normal configuration restore, the protected /etc/mosquitto tree was restored from the verified private bundle, but the Mosquitto executable/init script was absent because the original broker packages had lived in the previous writable overlay. opkg update completed with signed OpenWrt 24.10.7 feeds, then opkg install mosquitto-ssl mosquitto-client-ssl installed Mosquitto 2.0.18-r4 and dependencies successfully.

opkg detected that the restored /etc/mosquitto/mosquitto.conf differed from the package default and preserved the restored production file, placing the package default at /etc/mosquitto/mosquitto.conf-opkg. Treat this as expected/pass behavior. Next boundary: confirm /etc/config/mosquitto uses option use_uci '0', then enable/restart Mosquitto and verify loopback-only 127.0.0.1:1883.

Mosquitto local recovery PASS / bridge TLS pending - 2026-08-28

After reinstalling mosquitto-ssl and mosquitto-client-ssl, the restored static broker configuration is active with mosquitto.owrt.use_uci=0. Mosquitto is running as mosquitto -c /etc/mosquitto/mosquitto.conf and listens

only on 127.0.0.1:1883. The local MQTT Forwarder reconnected as gateway EUI 0016c001f139a1cb, so the local packet-forwarding-to-buffer path is restored.

The two restored bridges still target the old/test endpoint lora-test-server:8883 and currently report TLS errors. Treat this as a separate outbound bridge TLS issue, not as a failure of the custom image, modem drivers, local broker, or restored gateway configuration. Do not weaken verification with `bridge_insecure true`; isolate the TLS failure with the documented `openssl s_client mTLS/hostname` test.

Restored bridge failure reclassified as network reachability - 2026-08-28

The restored Mosquitto bridges still target the lab endpoint lora-test-server:8883. The direct `openssl s_client` test failed before TLS negotiation with `BIO_connect: Host is unreachable / errno=113`. The documented lab name maps lora-test-server to 192.168.8.50. Therefore the current blocker is IP reachability to the old lab server, not CA validation, certificate hostname validation, client-certificate rejection, or the custom Gateway OS image. Do not regenerate certificates or weaken TLS. Check the restored `/etc/hosts` mapping, gateway route, and reachability of 192.168.8.50 first.

Cloud MQTT cutover decision - 2026-08-28

The restored lora-test-server:8883 bridge is now retired from the active Phase 11 path. Do not spend additional time repairing the old lab VM merely to complete the custom-image cutover. The intended production gateway path remains local MQTT Forwarder -> loopback Mosquitto buffer -> public `mqtt.<DOMAIN>:8883`.

The cloud public MQTT endpoint is now commissioned as `smartagri-mqtt.duckdns.org:8883` through Reserved IPv4 129.212.208.168. The broker identity preserves its internal name and includes the public DuckDNS SAN; the real Gateway EUI client certificate completed strict hostname-verified mTLS, MQTT CONNECT, and an authorized command subscription. Do not point the gateway at a raw Droplet public IP.

When the physical gateway is reachable, change only the two Mosquitto bridge address lines from the retired lab endpoint to `smartagri-mqtt.duckdns.org:8883`, preserve the existing CA/client certificate/key and strict hostname verification, then prove buffered QoS 1 uplink delivery before making LTE the normal route. Reserved-IP reassignment/failover authority remains a separate HA acceptance item and does not block this normal-path cutover.

Post-build field LTE commissioning PASS - 2026-08-28

The custom image's modem support has now been proven beyond driver enumeration. On the physical gateway, `/dev/ttyUSB2` returned live SIM7600 AT responses, the SIM reported `+CPIN: READY`, the modem registered to DITO LTE (`+CEREG: 0,1, +COPS: ... "515 66 DITO",7`), and the DITO data APN `internet.dito.ph` was configured through OpenWrt logical interface `lte` using QMI on `/dev/cdc-wdm0`.

`ifup lte` created the dynamic `lte_4` child on kernel device `wwan0` and obtained IPv4 100.73.25.125/30 with carrier gateway 100.73.25.126. A temporary host route forced 1.1.1.1 through that LTE path and returned public ping replies with exit status 0; the temporary route was then removed. This proves that the custom Gateway OS build provides a working end-to-end SIM7600 QMI data path, not merely loadable kernel modules.

Do not duplicate the LTE commissioning commands here. The authoritative, repeatable field procedure is [Phase 11 section 11.9 - Verified SIM7600 / DITO QMI commissioning path](#).

That `peerdns=0` state was an early commissioning safeguard before the public MQTT/DNS path existed. Production has since advanced: keep `network.lte.defaultroute=0` so the health controller owns LTE route promotion, but use `network.lte.peerdns=1` so LTE can provide upstream DNS when Wi-Fi is unavailable. The image/build objective itself is complete; the reproducible overlay carries the current production policy.

Flash-ready AS923 + SIM7600 + journal release - 2026-09-01

The current accepted factory image supersedes the earlier modem-only candidate for new flashes. It is built from ChirpStack Gateway OS v4.12.0 commit 2112dbdbda48cd77ec1b82499e389abd728e84a1, OpenWrt b40dfac0a31695596f7c1f5f1519302ca8237f6e, target bcm27xx/bcm2709, and the proven DEVICE_rpi-2 profile.

```
file=chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz
bytes=28900364
sha256=bafe8b97baf9353df2654b1c8b71fa53d2ff764cd264d0ed6c924dd25a5ec67d
recovery=C:\Users\smartagriintern\lorawan-recovery\gateway-01\custom-v4.12.0-sim7600-as923-journal-20260901
```

Verified contents include RAK5146/SX1302 with AS923 / as923, AS923-1 923.2/923.4 MHz channel-file entries, SIM7600 serial/QMI support, uqmi, mosquitto-ssl 2.0.18-r4, ChirpStack Concentrator 4.7.1, MQTT Forwarder 4.6.0, and gateway-evidence 0.1.0-r2. Local Mosquitto listens only on 127.0.0.1:1883; a first-boot UCI-default creates/chowns its persistent data directory before s80mosquitto. The evidence writer is boot-enabled; the uploader is installed but disabled until its HTTPS/mTLS credentials are provisioned. No evidence TLS secrets are baked into the image.

The reproducible non-secret inputs are tracked under deployment/gateway/image-overlay/. The historical full SIM7600 .config size, following its symlink, is 283853 bytes; earlier 190216-byte notes are stale. The Windows recovery copy was hashed again after transfer and matched the build artifact byte-for-byte. Physical radio/modem/real-uplink checks remain post-flash hardware acceptance.

Physical clean-overlay commissioning PASS - 2026-09-02

The accepted artifact above was flashed to production Gateway-01 and its immutable SquashFS was independently re-extracted during troubleshooting. The image itself was clean of MQTT bridge secrets/configuration, Evidence mTLS credentials, and commissioning-only smartagri-* /etc/hosts overrides. The reused microSD initially exposed old copies of those files because stale writable OverlayFS state survived outside the compact factory-image write. This was corrected with a deliberate OpenWrt jffs2reset, followed by clean reprovisioning from protected off-device identities rather than by preserving the accidental overlay.

After the reset, factory-clean state was proved before secrets were copied. Gateway-01 was then secured, configured with permanent Ethernet management 192.168.20.11/24 plus Wi-Fi backhaul, restored with the authoritative purpose-separated MQTT/Evidence identities, and configured with strict public mTLS. RAK5146 reported Gateway EUI 0016c001f139a1cb; active region/channel plan remained AS923 / as923. Two MQTT bridge sockets were established to 129.212.208.168:8883, Evidence /readyz returned ready, writer/uploader services ran, and a controlled reboot preserved the full state. ChirpStack last_seen_at advanced after the clean reprovision and again after reboot (2026-09-02 01:30:01.955894+00).

Offline inspection also showed that this already-accepted immutable release contains MQTT Forwarder qos '0', despite the intended store-and-forward contract requiring QoS 1. The current Gateway-01 clean overlay explicitly persists QoS 1 and passed the cloud/reboot checks. The repository image overlay is corrected to QoS 1 for a future deliberate rebuild; **do not rebuild the working gateway merely to change this immutable default**. Use ../automation/commission-clean-overlay.sh for future clean post-flash commissioning.

A real gateway-evidence segment/receipt after the reset remains coupled to the first real sensor uplink, because gateway statistics alone do not create a LoRaWAN evidence record. Current SIM7600 non-enumeration is a separate physical-modem condition; the commissioned Wi-Fi gateway path is healthy and is not blocked by it.

Supporting runbook: 06-verify-gateway-os.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway 6. Verify Gateway OS, MQTT Buffering, and Integrity

This procedure verifies both gateway paths one layer at a time:

```
delivery -> MQTT Forwarder -> Mosquitto -> remote MQTT
evidence -> integrity journal -> hash chain -> cloud checkpoint/segment upload
```

Neither path is considered proven merely because its process is running.

A service showing as running is not enough. The checks must prove that:

- the RAK5146 is using the correct radio configuration;
- MQTT Forwarder publishes to the local broker;
- the local broker reaches the remote broker with the gateway certificate;
- uplinks remain queued during a WAN outage;
- the queue survives a gateway reboot;
- queued uplinks drain after connectivity returns;
- old downlink commands are not replayed after the outage;
- the journal records the same real uplink source independently of MQTT Forwarder;
- sequence and record/segment hashes remain continuous across normal operation and reboot;
- a WAN outage does not stop local journaling or create a false server checkpoint;
- recovery uploads missing segments and extends the last accepted off-device checkpoint;
- the server can reconcile journal evidence with the delivered remote gateway MQTT event.

Run the outage and reboot tests in staging or during an approved maintenance window.

Before you begin

Have these values ready:

```
Gateway address: <GATEWAY_IP>
Gateway EUI: <GATEWAY_EUI>
Region topic prefix: <REGION_TOPIC_PREFIX>
Remote broker: <MQTT_BROKER_FQDN>:8883
Expected queue free-space reserve: <BUFFER_FREE_SPACE_RESERVE_BYTES>
Journal implementation/version: <PINNED_JOURNAL_VERSION>
Journal storage budget: <JOURNAL_STORAGE_BUDGET>
Evidence ingest endpoint: https://<EVIDENCE_INGEST_FQDN>
```

For the real radio tests, use an approved OTAA device configured for the same region and channel plan as the gateway and ChirpStack server.

When the gateway or device has not yet been registered in ChirpStack, complete the relevant sections of [Register and test the gateway](#), then return to the traffic tests in this manual.

Step 1: Verify Gateway OS and network access

Run over SSH:

```
cat /etc/os-release
uname -a
date -u
ip addr
ip route
nslookup <MQTT_BROKER_FQDN>
monit status
ps w | grep '[m]osquitto'
```

Confirm that:

- the expected ChirpStack Gateway OS Base release is running;
- the UTC date and time are correct;
- the intended backhaul interface has an address;
- a default route exists;
- the remote broker name resolves to the expected address;
- Concentrator and MQTT Forwarder remain running;
- the local Mosquitto process is running;
- when the reviewed journal implementation has been installed, its service and uploader are healthy without repeatedly restarting.

Fix network or time problems before testing TLS. An incorrect clock can make a valid certificate appear expired or not yet valid.

For the commissioned Gateway-01 field topology, the normal production backhaul is the SIM7600 QMI interface `wwan0`; RJ45/`br-lan` at `192.168.20.11` is management-only and must not become the Internet default. After boot, allow the modem and one `lte-route-health` cycle to converge, then verify that the service is running, QMI has a current address/gateway, the metric-10 default is on `wwan0`, the production broker route resolves through `wwan0`, and UTC has synchronized. The controller intentionally retains a valid LTE default during a temporary broker/TLS failure so DNS/NTP and autonomous recovery remain possible.

```
ubus call network.interface.lte status
ubus call service list '{"name":"lte-route-health"}'
ip route show default
ip route get 129.212.208.168
date -u
logread | grep 'lte-route-health' | tail -n 20
```

On the 2026-09-15 reboot acceptance, DITO assigned a fresh PDP address, `lte-route-health` restored the metric-10 `wwan0` default automatically on its next cycle, the production MQTT mTLS probe passed, NTP corrected the clock, and the formal research-recorder preflight subsequently exited `0`. Treat a very early post-boot snapshot taken before the first successful health cycle as a startup observation, not as a routing failure; the route must appear automatically within the bounded controller cycle without a manual route command. The 2026-09-16 controller revision also distinguishes broker failure from LTE dataplane failure and performs staged autonomous recovery: bounded logical-interface restart first, then a cooldown-protected SIM7600 AT+CFUN=1,1 reset only if the

dataplane remains unavailable. Both stages were live-proven before the reboot-persistence check. The reboot-persistence check also passed: the daemon started from `S951te-route-health`, retained the repository script hash, used the 60-second boot grace instead of resetting the modem during QMI startup, then restored a fresh DITO PDP session, metric-10 default, and production mTLS without operator intervention.

Step 2: Verify the effective gateway configuration

Run:

```
logread -e chirpstack-concentratord | tail -n 100
uci show chirpstack-concentratord
uci show chirpstack-mqtt-forwarder
uci show chirpstack-udp-forwarder
```

Check the saved values against the setup:

- Concentratord uses the RAK5146 SX1302/SX1303 profile.
- The configured channel plan is the exact approved regional variant.
- The successful active SX1302/SX1303 startup log reports the expected 16-hexadecimal Gateway EUI; ignore inactive SX1301 gateway_id values.
- MQTT Forwarder uses the Concentratord backend.
- MQTT Forwarder connects to `tcp://127.0.0.1:1883`.
- MQTT Forwarder uses QoS 1 and the fixed `gw-fwd-<GATEWAY_EUI>` client ID when supported by the installed release.
- The MQTT topic prefix is `<REGION_TOPIC_PREFIX>`.
- UDP Forwarder has no enabled remote server.

Inspect both active broker files without displaying the private key:

```
grep -E
'^ (persistence|persistence_location|persistence_file|autosave_interval|max_queued_messages|max_queued_bytes|l
istener|protocol|allow_anonymous|include_dir)' \
/etc/mosquitto/mosquitto.conf

grep -E '^ (connection |address
|remote_clientid|cleansession|bridge_cacfile|bridge_certfile|bridge_keyfile|bridge_insecure|topic )' \
/etc/mosquitto/conf.d/bridge.conf
```

Confirm that:

- persistence is enabled;
- message and byte limits are finite;
- the listener is `1883 127.0.0.1`;
- both bridge addresses use `<MQTT_BROKER_FQDN>:8883`;
- bridge_insecure is false;
- event and state topics go out at QoS 1;
- command topics come in at QoS 0;
- the uplink bridge uses a persistent session;
- the downlink bridge uses a clean session.

Step 3: Verify queue storage and permissions

Run:

```
df -h /etc/mosquitto/data
mount
ls -ld /etc/mosquitto/data /etc/mosquitto/certs
ls -l /etc/mosquitto/data
ls -l /etc/mosquitto/certs
```

Pass this step only when:

- /etc/mosquitto/data is on persistent writable storage;
- the path is not under /tmp or another temporary filesystem;
- the mosquitto user can write the data directory;
- the private key is not world-readable;
- current free space is greater than the configured reserve.

The mosquitto.db file can be absent before Mosquitto has saved any persistent state. It should appear or change after the outage test creates queued messages.

Step 3A: Compare the gateway with the off-device security baseline

The buffer setup manual created <GATEWAY_EUI>-mqtt-security-baseline.txt on the administration workstation. This step independently recalculates the broker-configuration hash and certificate fingerprints and compares them with that approved off-device record.

On the gateway, create a fresh comparison file:

```
umask 077
{
  printf 'gateway_eui=%s\n' '<GATEWAY_EUI>'
  printf 'captured_utc=%s\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
  sha256sum /etc/mosquitto/mosquitto.conf /etc/mosquitto/conf.d/bridge.conf
  openssl x509 -in /etc/mosquitto/certs/<GATEWAY_EUI>.cert -noout -fingerprint -sha256 -serial -dates
  openssl x509 -in /etc/mosquitto/certs/ca.crt -noout -fingerprint -sha256 -subject -issuer
} > /tmp/gateway-mqtt-security-current.txt
cat /tmp/gateway-mqtt-security-current.txt
```

On the administration workstation, copy the current values:

```
scp root@<GATEWAY_IP>:/tmp/gateway-mqtt-security-current.txt \
./<GATEWAY_EUI>-mqtt-security-current.txt
```

Compare everything except the intentionally changing capture timestamp:

```
grep -v '^captured_utc=' ./<GATEWAY_EUI>-mqtt-security-baseline.txt \
> /tmp/gateway-mqtt-baseline.normalized

grep -v '^captured_utc=' ./<GATEWAY_EUI>-mqtt-security-current.txt \
> /tmp/gateway-mqtt-current.normalized

diff -u \
/tmp/gateway-mqtt-baseline.normalized \
/tmp/gateway-mqtt-current.normalized
```

Pass condition: diff prints nothing and returns exit status 0.

If it prints a difference, **stop gateway acceptance** and identify the exact changed configuration line or certificate. Do not overwrite the baseline just because the current gateway is online. For an intentional configuration change or certificate rotation, first complete the relevant setup procedure and all validation tests, document the approved change, then capture a new baseline from the known-good state.

Clean up temporary comparison files after review:

```
rm -f \
  /tmp/gateway-mqtt-baseline.normalized \
  /tmp/gateway-mqtt-current.normalized \
  ./<GATEWAY_EUI>-mqtt-security-current.txt
```

On the gateway:

```
rm -f /tmp/gateway-mqtt-security-current.txt
```

This baseline detects later configuration/certificate drift. It does not prove that `mosquitto.db` message contents were never modified by a privileged attacker.

Step 4: Verify the loopback MQTT listener

Run:

```
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

The listener must be:

```
127.0.0.1:1883
```

Fail this step when the listener is bound to `0.0.0.0`, `[::]`, a LAN address, or a WAN address. The local listener permits anonymous access only because it is restricted to loopback.

Step 5: Verify a local gateway event

Open an SSH session and subscribe to the local event topic:

```
mosquitto_sub -h 127.0.0.1 -p 1883 \
  -t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/#' -v
```

Generate one uplink from the approved test device.

A message must appear under the expected region prefix and Gateway EUI. The payload uses Protobuf and may not be readable as text. The topic and message arrival are the important checks.

Press `Ctrl+C` after the message arrives.

When no local event appears, troubleshoot Concentrator, the device channel plan, MQTT Forwarder, and the loopback broker before checking the remote TLS bridge.

Step 5A: Verify one journal record for the same real uplink

Complete this step only after the reviewed journal implementation described in [Gateway 4A](#) has been built, pinned, and installed.

Generate one fresh real uplink while observing both the local MQTT event and the journal's documented status/inspection interface.

Record, without exposing unrestricted payloads in ordinary logs:

```
Gateway EUI
journal sequence
boot ID
PHYPayload or approved PHYPayload digest
record hash
previous-record hash
current segment ID
```

Pass when:

- exactly one new journal sequence corresponds to the known uplink;
- the Gateway EUI is correct;
- the journal source is the supported Concentrator event interface, not a copy taken after Node-RED or ChirpStack;
- recomputing the versioned record hash produces the stored hash;
- the record references the preceding valid hash;
- MQTT Forwarder still produces the normal local event independently.

If the journal implementation has no safe command/API for inspecting and verifying a staging record, **stop deployment and add one to the reviewed implementation**. Do not parse undocumented binary files with an improvised production script.

Step 6: Verify the remote broker connection

Empty broker container stdout does not prove that no MQTT traffic arrived. Verify the topic directly first:

```
sudo timeout 45 docker exec mosquitto mosquitto_sub \
-h 127.0.0.1 -p 1883 \
-u chirpstack -P <CHIRPSTACK_MQTT_PASSWORD> \
-t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/stats' \
-v -C 1
```

A binary-looking Protobuf payload is normal. If this succeeds while ChirpStack still shows Last seen: Never, the gateway bridge path is proven and troubleshooting moves to ChirpStack region/backend configuration and exact EUI registration.

Then inspect application stdout if needed:

```
docker compose logs --since=5m --tail=200 mosquitto chirpstack
```

Confirm that:

- gw-up-<GATEWAY_EUI> connects with the gateway certificate;
- gw-down-<GATEWAY_EUI> connects with the same gateway identity;

- the broker accepts event and state writes only for this Gateway EUI;
- the broker allows command reads only for this Gateway EUI;
- ChirpStack receives the regional gateway event topic.

Also confirm in the ChirpStack web interface that the registered gateway's **last seen** time updates after the test uplink.

Verify cross-gateway isolation

When a second test Gateway EUI is available, attempt to publish or subscribe outside this gateway's allowed topic path using the first gateway certificate. The remote broker must deny the request.

Do not weaken the ACL when this test fails. Correct the certificate identity, topic path, or per-gateway ACL rule.

Step 7: Verify normal LoRaWAN traffic

Use the approved OTAA test device.

1. Power-cycle or reset the device so it performs a fresh join when appropriate.
2. Confirm that OTAA succeeds.
3. Send one uplink with a known payload or device frame counter.
4. Confirm that ChirpStack shows the expected Gateway EUI for the received uplink.
5. Confirm that the application receives one canonical record for the uplink.
6. Schedule one harmless Class A downlink.
7. Confirm that the device receives the downlink in the expected receive window.

QoS 1 can redeliver an MQTT message after a reconnect. The application or database must prevent one radio uplink from creating duplicate canonical records.

Step 8: Verify WAN-outage buffering

Disconnect only the gateway's WAN path. Keep the Raspberry Pi, RAK5146, Concentrator, MQTT Forwarder, and local Mosquitto running.

Before disconnecting WAN, run:

```
stat -c 'size=%s modified=%y' /etc/mosquitto/data/mosquitto.db 2>/dev/null || \
ls -l /etc/mosquitto/data

df -h /etc/mosquitto/data
```

Then perform the test:

1. Disconnect the gateway WAN or block only the route to the remote broker.
2. Keep an SSH session through a separate management path when possible.
3. Generate a known number of real uplinks.
4. Confirm that the local `mosquitto_sub` test still receives the gateway event topics.
5. Confirm that ChirpStack does not receive the outage uplinks yet.
6. Check `/etc/mosquitto/data/mosquitto.db` again. It should be created or increase after Mosquitto saves the queued state.
7. Confirm that free space remains above `<BUFFER_FREE_SPACE_RESERVE_BYTES>`.

8. Restore WAN connectivity.
9. Confirm that the uplink bridge reconnects.
10. Confirm that all expected uplinks reach ChirpStack and the application.
11. Confirm that the local queue stops growing after it drains.

The delivered MQTT count can include QoS 1 redelivery. Validate the application result using unique device identifiers and frame counters rather than assuming every MQTT delivery is unique.

Continuous queue growth after WAN recovery means the remote DNS, route, TLS, certificate, broker ACL, topic prefix, or ChirpStack subscription is still failing.

Step 8A: Verify the journal during WAN outage and recovery

Run this together with Step 8 so the same real test uplinks exercise both paths.

Before WAN loss, capture the latest accepted server checkpoint:

```
checkpoint gateway ID
checkpoint sequence
checkpoint record hash
checkpoint segment hash
server receipt ID/time
```

Then disconnect only WAN/backhaul and generate the known test uplinks.

During the outage pass only when:

- journal sequence continues increasing;
- the current segment continues to append/rotate within its configured budget;
- closed unuploaded segments remain local;
- the cloud checkpoint does **not** advance while the gateway cannot reach the evidence endpoint;
- Mosquitto and the journal use separate storage budgets and the OS free-space reserve remains safe.

After WAN returns, verify through the server-side procedures in [Gateway Integrity](#):

1. missing closed segments upload;
2. their first chain link extends the last accepted pre-outage anchor;
3. record and segment hashes verify;
4. the independently captured remote MQTT gateway events match the journal records;
5. a new checkpoint is accepted only after continuity is established;
6. unresolved missing evidence is reported as `evidence_gap` and contradictory evidence as `integrity_failure`.

Do not mark outage telemetry gateway-verified merely because Mosquitto successfully drained it.

Step 9: Verify queue persistence across reboot

Repeat this test while the WAN remains unavailable:

1. Generate several uplinks and confirm that the local queue grows.
2. Reboot the gateway cleanly:

```
sh reboot
```

3. Wait for Gateway OS to return.
4. Reconnect over SSH.
5. Confirm that Concentrator, MQTT Forwarder, and Mosquitto restart.
6. Confirm that `/etc/mosquitto/data/mosquitto.db` still exists and contains the queued state.
7. Generate another uplink while WAN is still unavailable.
8. Restore WAN connectivity.
9. Confirm that both the pre-reboot and post-reboot uplinks reach ChirpStack.
10. Confirm that the application retains one canonical record per real uplink.

The buffer is not persistent when messages queued before reboot disappear.

Step 9A: Verify journal continuity across reboot

Perform this during the same controlled WAN-outage reboot test when possible.

Before reboot record:

```
last journal sequence
last record hash
boot ID
open/last closed segment
```

After reboot:

1. verify the journal recovers the last valid complete record;
2. verify the boot ID changes;
3. verify the next sequence continues instead of resetting;
4. generate a fresh uplink;
5. verify its `previous_record_hash` references the last valid pre-reboot chain state;
6. restore WAN and prove the server accepts continuity across the reboot boundary.

Only an incomplete final/torn record may be discarded automatically. A previously complete record with an invalid hash is an integrity incident, not normal crash cleanup.

Step 10: Verify that stale downlinks are not replayed

The downlink bridge uses QoS 0 and a clean session so old commands are not intentionally retained.

1. Disconnect the gateway WAN.
2. Attempt one harmless test downlink through ChirpStack.
3. Allow the downlink and its receive window to expire.
4. Restore WAN connectivity.
5. Confirm that the expired downlink is not transmitted after recovery.
6. Send a new Class A downlink after the next uplink.
7. Confirm that the new downlink succeeds.

Stop and correct the downlink bridge when an expired command is transmitted after reconnecting.

Step 11: Verify queue limits without filling the SD card

Perform this only in staging.

1. Confirm the configured `max_queued_messages` and `max_queued_bytes` values.
2. Confirm the current free-space reserve.
3. Disconnect WAN.
4. Generate controlled test traffic until the configured queue limit is approached.
5. Confirm that storage use remains bounded.
6. Confirm that the filesystem does not become full or read-only.
7. Restore WAN and verify that retained messages drain.
8. Identify which messages were retained or dropped using device frame counters.

The correct behavior is bounded loss at the configured limit while preserving the gateway filesystem. An unlimited queue or a full SD card is a failed result.

Step 12: Verify backup and restore

Use a spare microSD card so the working gateway card remains untouched.

1. Flash the same approved Gateway OS Base factory image.
2. Restore the latest protected LuCI backup.
3. Confirm that the Mosquitto configuration and certificate files are present.
4. Confirm the private-key permissions.
5. Boot the restored gateway with the RAK5146 installed.
6. Confirm that the Gateway EUI and channel plan match the original gateway.
7. Confirm that both MQTT bridges authenticate.
8. Repeat one local uplink, one remote uplink, one safe downlink, and one short WAN-outage test.

A backup is not proven until it has restored a working gateway on separate media.

Troubleshooting by failing layer

Concentrator has no stable Gateway EUI

Return to [03-configure-concentrator.md](#). Check the RAK5146 profile, SPI assembly, power, and exact channel plan.

Local MQTT receives no gateway event

Check MQTT Forwarder's backend, loopback address, QoS, topic prefix, client ID, and packet filters. Do not change the remote broker yet.

Local events work but the remote broker receives nothing

Check gateway DNS, UTC time, bridge certificate files, certificate/key match, broker hostname, and remote ACL.

The remote broker receives events but ChirpStack does not update

Compare `<REGION_TOPIC_PREFIX>` with the regions enabled on the ChirpStack server. Confirm the exact Gateway EUI registration and the broker topic subscription.

The queue does not grow during an outage

Confirm that MQTT Forwarder publishes at QoS 1, the uplink bridge has `cleansession false`, persistence is enabled, and the persistence directory is writable by `mosquitto`.

The queue grows during normal connectivity

The bridge is not successfully delivering. Check route, DNS, time, TLS, certificate identity, broker ACL, and topic prefix in that order.

Queued messages disappear after reboot

Confirm that `/etc/mosquitto/data` is persistent, Mosquitto can write it, and the service stops cleanly during a normal reboot. Recheck the persistence settings and autosave interval.

An old downlink is transmitted after reconnecting

Confirm that the downlink bridge uses `cleansession true`, the command topic is `in 0`, and no second MQTT or UDP path is retaining commands.

Final acceptance

The gateway is ready for normal operation when all of these are true:

- Gateway OS Base boots with correct time and network access.
- Concentrator initializes the RAK5146 with the approved channel plan.
- The Gateway EUI remains stable.
- MQTT Forwarder publishes Protobuf messages to `127.0.0.1:1883` at QoS 1.
- Local Mosquitto listens only on loopback.
- Both remote bridges authenticate with the gateway certificate.
- The remote ACL restricts access to this Gateway EUI.
- The current broker configuration and certificate fingerprints match the approved off-device baseline.
- A real OTAA join, uplink, and safe Class A downlink succeed.
- Uplinks remain buffered during WAN loss.
- The queue survives a clean gateway reboot.
- Buffered uplinks drain after WAN recovery.
- Duplicate MQTT delivery does not create duplicate canonical application records.
- Expired downlinks are not replayed.
- Queue limits protect the SD card.
- A protected backup restores successfully to spare media.
- UDP Forwarder remains disabled.
- The journal independently records the supported Concentrator source path and record/segment hashes verify.
- Journal sequence/hash continuity survives a clean reboot.
- WAN loss preserves both the Mosquitto queue and local journal within separate finite budgets.

- Recovery uploads missing journal segments and extends the last accepted cloud checkpoint.
- Server reconciliation can distinguish verified, evidence_gap, and integrity_failure.
- Operators understand that the cloud-anchored history is stronger than a local checksum, but the software-only unanchored offline tail is not claimed to be unforgeable against full gateway root compromise.

Continue with [Gateway operations](#) for registration, routine tests, backup, recovery, and troubleshooting.

Current flash-image baseline - 2026-09-01

Use this exact factory artifact for the next clean flash:

```
file=chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz
bytes=28900364
sha256=bafe8b97baf9353df2654b1c8b71fa53d2ff764cd264d0ed6c924dd25a5ec67d
recovery=C:\Users\smartagriintern\lorawan-recovery\gateway-01\custom-v4.12.0-sim7600-as923-journal-20260901
```

Pre-flash acceptance already proved generated checksums, gzip/partition integrity, independent SquashFS extraction, RAK5146/SX1302 + AS923/as923 defaults, AS923-1 923.2/923.4 MHz channel files, SIM7600/QMI modules and uqmi, local Mosquitto, Concentrator/MQTT Forwarder, writer boot enablement, uploader default disablement, and no embedded gateway-evidence TLS secrets.

After flashing, validate only the hardware/runtime boundary: boot and writable overlay; RAK5146 Concentrator on AS923; SIM7600 tty/QMI enumeration; Mosquitto on 127.0.0.1:1883 with writable /etc/mosquitto/data; normal MQTT Forwarder enabled while UDP/mesh paths remain disabled; evidence writer running; uploader still disabled until mTLS provisioning; and one real uplink producing a journal record and reaching the telemetry path after protected bridge credentials are restored/provisioned.

Production MQTT bridge private keys/certificates and evidence-uploader mTLS material are intentionally not firmware contents. Keep them in the protected provisioning/recovery path.

Clean-overlay physical acceptance - 2026-09-02

Production Gateway-01 has now passed the post-flash hardware/runtime boundary after deliberately clearing stale writable OverlayFS state on the reused microSD. Independent extraction reconfirmed that the accepted SquashFS itself did not contain the stale MQTT/Evidence identities or commissioning /etc/hosts overrides. The gateway was reprovisioned from protected off-device credentials, rebooted, and reverified with:

Gateway EUI	0016c001f139a1cb
SX1302 model	rak_5146
region / channel plan	AS923 / as923
MQTT Forwarder	enabled, tcp://127.0.0.1:1883, qos=1
maintenance Ethernet	192.168.20.11/24, no default route
normal backhaul	Wi-Fi DHCP, default route via Wi-Fi
public MQTT	two established mTLS bridge sockets
Evidence	writer + uploader running, mTLS /readyz ready
commissioning host override	absent
ChirpStack last_seen_at	2026-09-02 01:30:01.955894+00 after reboot

Use `../automation/commission-clean-overlay.sh preflight` before provisioning a future newly flashed/reused card. If production keys/bridge state appear before provisioning, stop and investigate/reset stale writable overlay rather than assuming the immutable image contains secrets.

The accepted 2026-09-01 SquashFS was also found to contain MQTT Forwarder QoS 0. Runtime commissioning corrected Gateway-01 to QoS 1 and the tracked image overlay is corrected for a future rebuild. The current gateway does not require a rebuild because its clean persisted runtime has already passed reboot and cloud verification.

Real post-reset assembled EMU-01 LoRaWAN traffic has created and uploaded gateway-evidence segments/checkpoints successfully. The gateway writer/uploader, public mTLS ingest path, checkpoint/segment receipts, and cloud trusted-decoder verification are physically proven. At the 2026-09-02 powered-off acceptance boundary, the last EMU-01 application uplink was frame counter 666 at 05:31:45.795 UTC; Gateway-01 subsequently closed and receipted segment 28, journal/open was empty, and the cloud verifier reported all 492 selected v2 events verified with no evidence failures. This closes the Gateway OS physical evidence-lineage boundary; later sensor work should re-check it only if the gateway/evidence configuration changes.

Supporting runbook: 03-availability-tests.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Operations 3. Gateway Availability and Integrity Tests

Run these tests in staging or during a maintenance window where temporary telemetry interruption is acceptable. Capture timestamps, frame counters, queue database size, journal sequence/segment/checkpoint state, broker logs, verification status, and application row counts. Delivery evidence and integrity evidence answer different questions and must be measured separately.

Test 1: Normal reboot

reboot

Pass when Concentrator, MQTT Forwarder, local Mosquitto, and the reviewed journal services restart; Gateway ID is unchanged; the loopback listener remains private; both remote bridges reconnect; journal sequence/hash continuity resumes with a new boot ID; UDP Forwarder stays disabled; and a fresh uplink reaches ChirpStack.

Test 2: WAN interruption with buffering

1. Note `/etc/mosquitto/data/mosquitto.db` size and the last accepted device frame counter as the baseline.
2. Disconnect only WAN or 4G.
3. Keep RAK5146, Concentrator, MQTT Forwarder, and local Mosquitto running.
4. Generate a known number of real uplinks.
5. Confirm local publishes continue and the persistence database changes.
6. Confirm the remote broker receives no new events during the outage.
7. Restore WAN.
8. Verify all expected unique uplinks drain.
9. Confirm the journal continued sequencing throughout the outage and retained all closed unuploaded segments within its evidence budget.
10. Confirm the server checkpoint did not falsely advance while disconnected.
11. Confirm recovery uploads the missing journal segments and they extend the last pre-outage anchor.
12. Confirm the server correlates the recovered journal records with the drained gateway MQTT events.

Compare disconnect detection, bridge retry interval, outage duration, queued count, drain duration, duplicate MQTT deliveries, unique application records, and stale/offline transition with the outage design. Queue growth without later drain means the remote path is still unhealthy; more application rows than unique events means deduplication is incomplete.

Test 3: Gateway reboot during WAN outage

Buffer real uplinks while WAN is unavailable, then reboot the gateway cleanly.

Pass when:

- the Mosquitto queue survives reboot and drains after WAN recovery;
- the journal continues from the last valid complete record rather than resetting;
- a new journal boot ID appears;
- the first post-reboot record links to the last valid pre-reboot chain state;
- server reconciliation accepts the chain across the reboot boundary.

Queue loss is an availability failure. Journal chain reset/corruption is an integrity failure. Keep those diagnoses separate.

Test 4: Remote broker restart

```
cd /opt/chirpstack-docker
docker compose restart mosquitto
docker compose logs -f mosquitto chirpstack
```

Pass when the local queue retains events during the restart, both bridge connections reconnect automatically, queued event/state traffic drains, and a later fresh downlink succeeds.

Test 5: ChirpStack restart

Restart ChirpStack while leaving both brokers available. Verify remote MQTT remains healthy, events are processed after ChirpStack reconnects, and any duplicates remain idempotent downstream.

Test 6: Invalid gateway certificate

In staging, replace only the bridge certificate with an untrusted or mismatched test certificate.

Expected:

- remote broker rejects both bridge connections;
- local event/state messages accumulate in the finite queue;
- no gateway event reaches ChirpStack;
- restoring the valid certificate drains the queue;
- cross-gateway access remains denied.

Use a verified rollback bundle.

Test 7: Queue limit and storage pressure

Generate enough staging messages to approach, but not exhaust, the configured queue limits.

Verify warnings or drops are visible, the free-space reserve remains intact, the filesystem stays writable, and the runbook explains how to identify which messages were lost. Silent drops or a read-only filesystem are failures even when the broker process remains running.

Test 8: Stale-downlink prevention

During WAN loss, create only a non-hazardous test downlink. It must fail or expire and must not transmit after recovery. A new downlink created after recovery must work.

Test 9: Remote endpoint failover

Test the exact load-balancer or broker design. A load balancer does not create shared Mosquitto sessions or queue state. Measure reconnect, loss, duplicates, and certificate handling.

Test 10: Journal tamper and gap fixtures

Use **copies** of closed staging segments, never the only real evidence object.

Test separately:

1. change one byte in a copied complete record;
2. remove one complete record;
3. swap two complete records;
4. present an older segment/checkpoint lineage than the server's latest accepted anchor.

Pass when the verifier rejects the changed/reordered/rollback evidence. A missing required whole segment with no contradictory bytes is `evidence_gap`; a proven conflicting hash/payload is `integrity_failure`.

Test 11: Journal storage pressure

Reduce the staging evidence budget enough to approach warning/critical thresholds without filling the OS filesystem.

Pass when the emergency reserve survives, alarms are visible, unuploaded evidence is not silently overwritten, and affected events are explicitly classified as an evidence gap if retention is exhausted.

Final acceptance

- Uplink queue survives WAN loss and gateway reboot.
- Queue drains within the target and application rows remain unique.
- Queue limits protect storage.
- Invalid certificates and cross-gateway topics are rejected.
- Stale downlinks are not replayed.
- No UDP fallback starts.
- Journal sequence and hash continuity survive normal operation and reboot.
- Cloud checkpoints anchor connected history and do not falsely advance while disconnected.
- Recovery uploads missing segments and reconciles them with drained MQTT events.
- Tamper, deletion, reorder, rollback, and storage-gap fixtures produce the documented verification outcomes.
- The residual full-root/offline software-only limitation is recorded rather than hidden.